

General Non-Clairvoyant KV-Cache Scheduling via Regime-Aware Routing

Yiding Feng^{*} Siyu Liu[†] Zonghan Yang[‡] Yuhao Zhang[§]

Abstract

We study non-clairvoyant scheduling for batched Large Language Model (LLM) inference under a hard Key-Value (KV) cache memory budget. Each request has a known prompt length but an unknown response length, and its memory footprint comprises a fixed prompt component together with a response component that grows with each decoded token. At each decoding round, the scheduler chooses a feasible batch of active requests; evicting a request discards its accumulated cache states, wasting prior computation. The goal is to minimize total completion time against the optimal clairvoyant schedule that knows all response lengths.

We present the first $O(1)$ -competitive algorithm for arbitrary prompt lengths and arbitrary response lengths with no additional assumptions. Rather than relying on a single universal scheduling policy, our algorithm is built on a novel *regime-aware routing* framework. Specialized sub-schedulers handle different memory-growth geometries, while a meta-scheduler time-shares the memory budget across them and dynamically routes each job as its execution progressively reveals its behavior. This framework also yields constant-competitive guarantees for makespan and for total completion time under online arrivals.

1 Introduction

Large Language Model (LLM) inference has become a cornerstone of modern computing infrastructure, supporting interactive assistants, automated workflows, and agentic applications at massive scale. As these workloads shift from short exchanges toward longer and more computation-intensive tasks, the inference phase has become a major contributor to operational cost and energy consumption [Luccioni et al., 2024, Stojkovic et al., 2025a,b]. Meeting quality-of-service requirements under these resource pressures calls for careful scheduling across the serving stack, from cluster-wide request routing to the memory management logic on individual serving instances.

This work focuses on replica-level scheduling inside an LLM inference service. In modern LLM serving stacks, a routing layer assigns each incoming request to one of many parallel serving instances, known as *replicas*; within each replica, the local scheduler decides which requests to process together and how to manage the finite memory budget [Yu et al., 2022, Kwon et al., 2023, Zheng et al., 2024, Sun et al., 2024]. The central challenge arises from the Transformer architecture, whose self-attention mechanism stores intermediate states in a per-request *Key-Value (KV) cache* to avoid quadratic recomputation during decoding [Vaswani et al., 2017, Pope et al.,

^{*}Hong Kong University of Science and Technology. Email: ydfeng@ust.hk

[†]Shanghai Jiao Tong University. Email: williamliusy@sjtu.edu.cn

[‡]Shanghai Jiao Tong University. Email: fstqwq@sjtu.edu.cn

[§]Shanghai Jiao Tong University. Email: zhang-yuhao@sjtu.edu.cn

2023]. Processing a request involves a *prefill* phase that encodes the input prompt and initializes the cache, followed by a *decode* phase that generates response tokens autoregressively, appending new KV entries at each step. The memory footprint of a request therefore has a fixed component from the prompt and a component that grows linearly with the response length. Modern systems manage this dynamic growth through two mechanisms: *continuous batching* [Yu et al., 2022], which updates the active batch at the granularity of individual decoding iterations, and *block-based cache management* [Kwon et al., 2023, Zheng et al., 2024], which allocates and recycles memory across requests. When the budget is exhausted, active requests must be *killed*, discarding the KV-cache entries accumulated during decoding. The scheduler must therefore balance high concurrency, which improves throughput, against the cost of kills that waste prior computation.

This tension is compounded by a second critical difficulty: the scheduler is *non-clairvoyant*. Response lengths are not known in advance but are determined by the model’s generation process and revealed only at completion. The autoregressive and stochastic nature of text generation makes exact prediction inherently impossible, even with dedicated predictors [Zheng et al., 2023, Fu et al., 2024, Qiu et al., 2024]. In classical non-clairvoyant scheduling, a job of unknown size occupies a fixed amount of resource whenever it runs; here, uncertainty in response length directly translates into uncertainty in future memory demand, because the cache grows with every decoded token. The scheduler must therefore manage dynamically growing resource requirements under incomplete information—a doubly constrained trade-off between latency and memory safety.

This places the problem in the non-clairvoyant setting of scheduling theory [Motwani et al., 1994, Becchetti and Leonardi, 2004], with the additional feature that the resource demand of a job increases during its execution. For this problem, a recent line of theoretical work [Jaillet et al., 2025, Wang et al., 2025, Chen et al., 2025b, Feng et al., 2026, Kong et al., 2026] has established guarantees for the single-replica model under restricted regimes, such as clairvoyant scheduling, identical prompt lengths, or large-memory settings. Removing these restrictions, we ask:

Can a non-clairvoyant scheduler perform comparably to the optimal schedule computed with full knowledge of all response lengths?

To make this question concrete, we adopt the standard offline batch model studied in recent theoretical work on LLM inference scheduling [Wang et al., 2025, Chen et al., 2025b, Feng et al., 2026, Kong et al., 2026]. A single serving instance, or replica, has a hard memory budget M . A set of jobs (inference requests) arrives at time zero. Each job has a known prompt length s_i and an unknown response length o_i . Time proceeds in discrete rounds. In each round, the scheduler chooses a batch of unfinished jobs to process; every job in the batch decodes one token. The memory consumed by a job equals its prompt length plus the number of tokens it has already decoded in its current uninterrupted attempt. The total memory of all active jobs must not exceed the budget M . The scheduler may also kill an active job, discarding all progress from that attempt and forcing a restart. The objective is to minimize total completion time—the sum of the completion times of all jobs—measured against an optimal clairvoyant schedule that knows all response lengths in advance. A non-clairvoyant algorithm is *competitive* if its total completion time is at most a constant factor times this optimum on every instance.

Compared to classical non-clairvoyant scheduling, the defining feature of our model is that resource demands are dynamic rather than static. A batch that is feasible at the start of a round may become infeasible as its members grow, forcing the scheduler to kill jobs to free memory or to run fewer jobs in parallel.

1.1 Our Results and Techniques

We answer the aforementioned question affirmatively. In a nutshell, we give the *first constant-competitive non-clairvoyant* scheduling algorithm for the offline batch model with arbitrary prompt lengths and arbitrary response lengths, requiring no additional assumptions on the problem instances. Formally, our main result is as follows.

Theorem 1. *For every feasible batch KV-cache scheduling instance $\mathcal{J}$ with arbitrary prompt lengths and arbitrary response lengths, there exists a fully non-clairvoyant scheduling algorithm whose total completion time is within a constant factor of the optimal clairvoyant total completion time:*

$$\text{ALG}(\mathcal{J}) = O(1) \cdot \text{OPT}(\mathcal{J}).$$

Beyond satisfying the competitive guarantee stated in the theorem, our algorithm runs in polynomial time and is also constant-competitive for the alternative objective of makespan.¹

In the remainder of this section, we give an overview of the key challenges and the main technical ingredients of our algorithm: *regime decomposition*, which partitions jobs into geometric regimes; *budget-doubling*, which handles unknown response lengths within each regime; and a *routing meta-scheduler*, which combines the regimes with only constant-factor overhead.

From area order to regime decomposition. A natural starting point is the clairvoyant setting, where the scheduler knows all response lengths in advance. Following prior work [Feng et al., 2026], we define the (*memory-time*) *area* of a job as its total memory occupation over a successful run, i.e., $A_i \triangleq s_i \cdot o_i + o_i(o_i + 1)/2$. Below, we refer to memory-time area simply as *area*. Note that the optimal total completion time can be lower bounded by sorting all jobs by nondecreasing area and charging each job for the cumulative area that must be supplied before it can finish: the r -th smallest job cannot complete before the system has delivered the total area of the first r jobs, which takes time proportional to the sum of their areas divided by the memory budget (see Lemma 1). This bound holds because the system can supply at most M units of memory-time area per round, so the aggregate area of all jobs must be paid for in time.

The area lower bound not only provides a tractable way to characterize the optimal clairvoyant benchmark; more importantly, it also suggests a sufficient condition for a constant approximation or competitiveness:

- **(High memory utilization)** the schedule should keep a constant fraction of the memory occupied; and
- **(Area-order completion)** the completion order should be close to nondecreasing memory-time area order.

One might hope to achieve both with a single priority rule over job types—for example, greedy scheduling by area.² Indeed, all known non-clairvoyant algorithms in the related restart and KV-cache scheduling models, and the majority of clairvoyant KV-cache scheduling algorithms, derive their schedules from a global ordering criterion [Jäger et al., 2025, Feng et al., 2026, Jaillet et al.,

¹In our model, makespan is a relatively easier objective; we therefore focus primarily on completion time (i.e., latency), which is also the more practically important metric in LLM inference.

²Specifically, put all jobs in a priority queue based on their area. At any time step, if the job at the top of the priority queue can be added to the current batch while respecting the memory constraint, remove it from the queue and add it to the processing batch.

2025, Chen et al., 2025b, Kong et al., 2026]. However, we show that this is impossible even with full knowledge of all response lengths in Proposition 1. The obstruction has two complementary forms. If the scheduler prioritizes small-area jobs, it may leave the memory severely underutilized while a long small-prompt job monopolizes the batch. If it prioritizes large-area jobs, it may block many small-area jobs behind a few large-prompt jobs that must run serially. These two failure modes impose opposite pressures: high utilization and area-consistent ordering cannot be reconciled by one global priority. This motivates a decomposition into geometric regimes that are treated separately.

We partition the jobs using a prompt cutoff $\Lambda = M/4$ into three regimes:

- **(Large jobs)** $s_i > \Lambda$. These jobs are so wide that each one nearly fills the memory budget alone.
- **(Small prompt-heavy jobs)** $s_i \leq \Lambda$ and $o_i \leq s_i$. The response is no longer than the prompt, so the memory footprint stays within a constant factor of the prompt length.
- **(Small response-heavy jobs)** $s_i \leq \Lambda$ and $o_i > s_i$. The memory footprint grows well beyond the prompt, and the area is dominated by the response length.

This three-way partition is the regime decomposition used by the algorithm: jobs are classified by their geometric properties and sent to distinct scheduling pipelines, each specialized to a particular memory-growth profile.

Each regime, taken in isolation, admits a constant-competitive non-clairvoyant algorithm, which we describe in the next two parts. The remaining challenge is that the non-clairvoyant scheduler does not know which regime a job belongs to, because the distinction between prompt-heavy and response-heavy depends on the unknown response length. We explain how to resolve this difficulty when we discuss the meta-scheduler below.

Rectangle scheduling for large and prompt-heavy jobs. For large jobs and prompt-heavy jobs, the KV-cache footprint grows by at most a constant factor during execution. Specifically, if $o_i \leq s_i$, then the memory usage at every timestep lies between $s_i + 1$ and $2s_i$. This allows us to abstract each job as a rectangle of fixed width $w_i \triangleq 2s_i$ and unknown processing time $p_i \triangleq o_i$, where killing an active attempt discards all progress.³ To our knowledge, this *non-clairvoyant* version of rectangle strip scheduling has not been studied in the literature, and our results here may be of independent interest.

Our scheduler RECT (Algorithm 1) maintains a priority queue of attempts ordered by their *area budget* $w_i \cdot 2^r$, where 2^r is the geometrically increasing length cap for the r -th attempt of job i . At each event time, the scheduler greedily admits the minimum-budget attempt if it fits in the remaining memory. If an attempt reaches its cap without completing, it is killed and replaced by a doubled attempt, which is added back to the priority queue with the updated area budget.

The analysis rests on a *queue monotonicity* (Lemma 4): the minimum area budget among pending attempts is weakly increasing over time, and every active attempt has area budget no larger than every pending attempt. This invariant lets us split the completion time of each job into active time and waiting time. The active time—the total time during which some (possibly unsuccessful) attempt of job i is executing, so that no attempt of that job is waiting in the queue—is bounded by $4p_i$ because the geometric caps up to and including the first successful one sum to less than $4p_i$. The waiting time—the total time during which an attempt of job i is in the priority

³Here we use terminology from strip packing and resource-constrained scheduling [Lodi et al., 2002, Jansen and Rau, 2021].

queue—is controlled by a utilization bound: when all rectangle widths lie in a bounded range $[\delta, \Delta]$, every waiting round has active memory of at least $\max\{\delta, M - \Delta\}$. For large jobs, $\delta = 2\Lambda = \Omega(M)$, so utilization is constant, as desired; for prompt-heavy jobs, $\Delta = 2\Lambda$, and utilization is at least $M - 2\Lambda = \Omega(M)$, again as desired. Moreover, the queue monotonicity ensures that the completion order approximately follows the area order. Consequently, the scheduler RECT (Algorithm 1) is 36-competitive when all jobs are large, and also when all jobs are small prompt-heavy.

Remarkably, even a few small response-heavy jobs suffice to break the guarantee: memory may still be well utilized, but jobs can now finish in the wrong order—with large-area jobs completing before small-area ones—because the queue monotonicity no longer holds. This ordering failure makes the competitive ratio unbounded.

Geometric slicing for response-heavy jobs. When the response length dominates the prompt, the memory footprint grows far beyond the prompt length, so the fixed-width rectangle view no longer applies. Instead, the area of a response-heavy job is governed by its response length: for each job i , the area $A_i \triangleq s_i \cdot o_i + o_i(o_i + 1)/2$ lies between $o_i^2/2$ and $2o_i^2$. Thus response-length order and area order agree up to a constant factor.

We handle these jobs with a scheduler RESPONSE (Algorithm 2) that extends the geometric-slicing approach of Feng et al. [2026] from identical to non-identical prompt lengths. The algorithm proceeds in phases $r = 0, 1, \dots$ with geometrically increasing response caps $\tau_r = 2^r$. In each phase, every unfinished job with $s_i \leq \tau_r$ becomes *eligible*. All eligible jobs are then scheduled using the SPS (Staggered Pipeline Scheduling) subroutine of Feng et al. [2026], which packs jobs with identical prompt and response lengths into a high-utilization batch. We invoke SPS by treating all eligible jobs as sharing a common proxy prompt $\bar{s}_r = \min\{\tau_r, M - \tau_r\}$ and the current cap τ_r as their common response length.

The key insight is that within a phase, the shared proxy prompt lets SPS guarantee high memory utilization. Moreover, since the caps grow geometrically, each job completes in the first phase whose cap exceeds its true response length. This means the completion order induced by the phases approximately follows the response-length order, which in this regime is close to the area order. Consequently, the scheduler is $236/3 \approx 78.67$ -competitive when all jobs are response-heavy.

Remarkably, even a few prompt-heavy jobs are enough to break the guarantee. If we keep the same phase-based eligibility (i.e., admitting a job whenever its prompt fits under the current response cap), the SPS schedule can leave memory underutilized because prompt-heavy jobs consume far less cache than their proxy length suggests. In addition, response-length order and area order can diverge dramatically for such jobs: a short-response job with a long prompt may have large area, while a long-response job with a short prompt may have small area. Either mismatch alone makes the competitive ratio of our scheduler RESPONSE (Algorithm 2) unbounded.

Meta-scheduling with routing. So far, we have described separate schedulers for large jobs, small prompt-heavy jobs, and small response-heavy jobs. From an engineering perspective, if we had three replicas—each with its own memory budget of M —we could route each regime to a dedicated replica and immediately obtain an $O(1)$ -competitive guarantee, provided the jobs were partitioned correctly. The remaining challenge is to achieve the same guarantee on a single replica, without knowing the partition in advance.

If the three-way partition were known, we could combine the sub-schedulers via a black-box meta-scheduler. The meta-scheduler BLACKBOXMETA (Algorithm 3) runs the sub-schedulers in round-robin order over doubling stages of length 2^r . Each sub-scheduler receives a fresh copy of its job set in each of its allotted rounds. Completed jobs are treated as dummy jobs in later stages.

A per-job completion bound shows that this time-sharing loses only a constant factor relative to running each sub-scheduler in isolation.

In the non-clairvoyant setting, however, the partition is not known: for a job with small prompt, the scheduler cannot tell whether $o_i \leq s_i$ or $o_i > s_i$ without running it. We therefore use a routing meta-scheduler ROUTE (Algorithm 4). Small jobs are initially assigned to the prompt-heavy branch. If a job completes before generating s_i tokens, it is finished; if it reaches s_i tokens without completing, it is *certified* as response-heavy and routed to the response-heavy branch. Large jobs are sent directly to the large-job branch.

The routing meta-scheduler uses the same doubling stages as the black-box version. In each stage, it runs the prompt-heavy branch, then the response-heavy branch (which now includes any newly certified jobs), then the large branch. The key observation is that a job certified in the prompt-heavy branch during stage r is available to the response-heavy branch in the same stage, and the doubling lengths ensure that both certification and completion are accommodated within a constant factor of their isolated completion times (Lemma 11). Combining the three branch guarantees yields the final $O(1)$ -competitive guarantee for all instances with arbitrary prompt lengths and arbitrary response lengths stated in Theorem 1.

Variants and extensions. Beyond our main non-clairvoyant result stated in Theorem 1, we establish several complementary guarantees using our algorithmic framework to chart the broader landscape of KV-cache scheduling. First, in the clairvoyant setting, our framework yields a 16-approximation algorithm for all instances with arbitrary prompt lengths and arbitrary response lengths (Theorem 6), and a $(3 + o(1))$ -approximation for the large-memory regime where every job satisfies $s_i + o_i = o(M)$ (Corollary 2).⁴ Second, the same routing algorithm yields an $O(1)$ -competitive ratio for the makespan objective (Theorem 8), showing that the structural benefits of regime decomposition extend beyond total completion time. Third, we extend our results to online arrivals, where jobs arrive over time and reveal their prompt lengths upon arrival: we give an online variant of our routing meta-scheduler that remains $O(1)$ -competitive for total completion time (Theorem 9).⁵

System insights. Modern LLM inference systems already allow global request routing across different replicas. Our results provide a principled foundation for how this routing should be done: partition jobs according to their geometric structure—large jobs, small prompt-heavy jobs, and small response-heavy jobs. This partition can be implemented at the global router level, where prompt lengths are known and response-based classifications can be updated dynamically as execution reveals job behavior. Once jobs are routed to separate replicas, each local scheduler can simply run the corresponding sub-scheduler from our framework. This avoids the overhead of the theoretical meta-scheduler, which must time-share a single memory budget across heterogeneous classes. Consequently, such an implementation achieves a better competitive ratio than the single-replica theoretical bound, while remaining easy to deploy.

The geometric regime decomposition explains why the existing separation between routing and local scheduling is algorithmically useful. Simple local rules, properly routed by geometric class, can achieve both worst-case robustness and practical efficiency.

⁴The same approximation factor of 3 for large-memory instances has been established in concurrent work by Kong et al. [2026].

⁵For the standard flow-time objective under adversarial arrivals, strong impossibility results hold: even restricted settings admit super-constant lower bounds on the competitive ratio [Jaillet et al., 2025]. Total completion time is therefore the natural latency objective for constant-competitive guarantees in the online non-clairvoyant setting.

1.2 Organization

Section 2 defines the scheduling model and develops an area-based lower bound on the optimal completion time. Section 3 classifies jobs into three regimes—large, prompt-heavy, and response-heavy—and motivates treating them separately through explicit hard instances. Section 4 gives the rectangle scheduler RECT (Algorithm 1), which handles large and prompt-heavy jobs by exploiting their nearly constant memory footprint, while Section 5 gives the response scheduler RESPONSE (Algorithm 2), which handles the growing footprint of response-heavy jobs through area-geometric slicing with doubling response caps. Section 6 presents the black-box meta-scheduler BLACKBOXMETA (Algorithm 3) that time-shares the memory budget across independent sub-schedulers, and extends it to the routing meta-scheduler ROUTE (Algorithm 4) that resolves the unknown job partition during execution, completing the proof of Theorem 1. Finally, Sections A to D contain the clairvoyant approximation guarantees, deferred analyses for the rectangle and response-heavy schedulers, and extensions to makespan and online-arrival completion time.

1.3 Related Work

KV-cache scheduling. Jaillet et al. [2025] initiate the theoretical study of KV-cache scheduling for LLM inference by formulating the scheduling model, proving an $\Omega(\sqrt{n})$ lower bound for deterministic algorithms under adversarial online arrivals, and giving the Memory-Constrained Shortest-First rule which achieves a 9216-approximation in the offline identical-prompt clairvoyant setting. In the clairvoyant setting, Wang et al. [2025] study the large-memory regime with heterogeneous prompt and response lengths and give a polynomial-time $(48 + \varepsilon)$ -approximation for any fixed $\varepsilon > 0$. Recently, concurrent work by Kong et al. [2026] improves the approximation ratio for the clairvoyant large-memory regime to $3 + o(1)$. For non-clairvoyant settings, Chen et al. [2025b] study scheduling under output-length uncertainty and give an $O(\log M)$ -competitive algorithm in the large-memory regime. Feng et al. [2026] give a non-clairvoyant geometric-slicing algorithm for the identical-prompt offline batch model, with a competitive ratio of at most 61.92 in general and 32 in the large-memory regime. Parallel lines of work [Ao et al., 2025, Dai et al., 2025] study stochastic arrivals through fluid-guided online scheduling and queueing-throughput viewpoints with different optimization objectives.

Non-clairvoyant scheduling. Classical non-clairvoyant scheduling is mainly studied in a pre-emptive pause-and-resume model, where processing times are unknown until completion and a suspended job keeps its completed work. Motwani et al. [1994] introduced this model and gave deterministic and randomized upper and lower bounds; Becchetti and Leonardi [2004] developed randomized multilevel-feedback algorithms for total flow time on single and parallel machines. Later variants add limited side information, including approximate processing-time classes [Becchetti et al., 2004], job-size predictions [Im et al., 2023], predictions for only a subset of jobs [Benomar and Perchet, 2024], and progress-bar feedback [Benomar et al., 2025]. Im et al. [2018] introduce the packing/polytope scheduling framework, where active jobs receive processing rates subject to packing constraints on the rate vector, and give non-clairvoyant algorithms including a constant-competitive guarantee for total weighted completion time. Jäger et al. [2026] sharpen the proportional-fairness analysis for this framework, including important monotone special cases, under the same total weighted completion-time objective.

Another line of research studies restart models, where interrupting a job destroys its accumulated progress. In clairvoyant online settings, restarts have been studied for single-machine total

completion time and for completion-time and flow-time lower bounds [van Stee and Poutré, 2005, Epstein and van Stee, 2003]. Jäger et al. [2025] study single-machine scheduling with a weighted completion-time objective in a non-clairvoyant restart model, prove lower bounds for deterministic restart strategies, and give tight analyses for geometric restart strategies.

Completion-time scheduling. Total and weighted completion time are central min-sum objectives in scheduling. For the basic single-machine weighted case, Smith [1956] gives an exact ordering principle by sorting jobs according to processing time per unit weight. For richer machine environments, Hall et al. [1997] develop general offline and online approximation techniques for completion-time objectives, using linear-programming relaxations to guide list scheduling. Chekuri et al. [2001] give ordering and rounding methods for average completion time with release dates, parallel machines, and precedence constraints; their algorithms derive an order from a preemptive or linear-programming relaxation and round it into a nonpreemptive schedule. More recently, Li [2020] shows the strength of time-indexed linear-programming relaxations for total weighted completion time, including precedence-constrained machine scheduling; using completion-time structure as a guide, Chen et al. [2025a] give a gradient-descent meta-algorithm based on the residual weighted completion-time optimum and obtain scalable algorithms for weighted flow time.

Geometric strip packing & scheduling. The memory-time view represents executions as growing geometric regions packed under a fixed memory width. The KV-cache regions in this model are trapezoidal, while related geometric-packing literature studies rectangles and more general polygons, often with objectives such as the number of bins or the strip length. Lodi et al. [2002] survey two-dimensional packing models, including bin packing and strip packing variants. Adamaszek et al. [2017] prove APX-hardness for strip packing, including the case with polynomially bounded input data. Steinberg [1997] gives a 2-approximation for rectangle strip packing. Jansen and Zhang [2007] give a $(3 + \epsilon)$ -approximation for maximum-profit rectangle packing in a bounded rectangle. More general polygon packing has also been studied: Alt et al. [2017] give approximation algorithms for packing convex polygons into minimum-area containers, and Kurpisz and Suter [2023] improve approximation guarantees for translational packing of convex polygons, including polygon strip packing.

Resource-constrained scheduling gives a closely related strip view: a job with processing time and resource demand occupies a rectangle in the time-resource strip, so makespan corresponds to strip length and total completion time corresponds to the sum of rectangle endpoint times. Under this view, Jansen and Rau [2021] give a tight $(3/2 + \epsilon)$ -approximation for the unweighted makespan objective on identical machines with one shared renewable resource, while Cohen et al. [2024] give constant-approximation algorithms for the weighted total completion-time objective on capacitated parallel machines, using volume-by-weight ordering and a hybrid treatment of large resource demands.

2 Preliminaries

In this section, we define the scheduling model, the non-clairvoyant information structure, the total completion-time objective, and the benchmark bounds used throughout the paper. Our notation mostly follows the offline batch setting studied in the recent line of work on LLM inference scheduling [Jaillet et al., 2025, Feng et al., 2026].

Jobs and scheduling actions. The system has a single serving instance with a KV-cache

memory budget $M \in \mathbb{N}^+$. There are n jobs, denoted by $\mathcal{J} \triangleq [n]$. Job $i \in \mathcal{J}$ has a known prompt length $s_i \in \mathbb{N}^+$ and an unknown response length $o_i \in \mathbb{N}^+$. We assume that each job is individually feasible:

$$s_i + o_i \leq M, \quad \forall i \in \mathcal{J}.$$

All jobs arrive at time zero. Time is discrete and proceeds in rounds $t = 0, 1, 2, \dots$, where time t denotes the beginning of round t . At time t , each unfinished job is either active in a current attempt or inactive. For each active job i , let $u_{i,t}$ denote the number of response tokens already decoded in its current uninterrupted attempt before round t begins. For notational convenience, define $u_{i,t} = 0$ when job i is inactive. At the beginning of each round t , the scheduler chooses an active batch $\mathcal{B}_t \subseteq \mathcal{J}$ of unfinished jobs. Every job $i \in \mathcal{B}_t$ receives one unit of processing during the round. If $u_{i,t} + 1 = o_i$, then job i completes at the end of round t , and its completion time is $t + 1$. Before deciding $\mathcal{B}_t$, the scheduler may *kill* any active unfinished job i , discarding all progress in its current attempt and setting $u_{i,t} = 0$; every unfinished job in $\mathcal{B}_{t-1} \setminus \mathcal{B}_t$ is implicitly killed.

KV-cache memory feasibility. If job i is active in round t , its KV-cache memory usage is

$$s_i + u_{i,t} + 1,$$

where the term $+1$ accounts for the token being decoded in the current round. A schedule is feasible if every active batch satisfies

$$\sum_{i \in \mathcal{B}_t} (s_i + u_{i,t} + 1) \leq M, \quad \forall t \geq 0.$$

In other words, the total memory occupied by all active jobs (i.e., their fixed prompts $\sum_{i \in \mathcal{B}_t} s_i$ plus the KV entries generated so far $\sum_{i \in \mathcal{B}_t} u_{i,t}$ plus the tokens currently being decoded $|\mathcal{B}_t|$) must never exceed the memory budget M .

Non-clairvoyant setting and competitive analysis. The non-clairvoyant scheduler knows the number of jobs n , memory budget M , and the prompt lengths $\{s_i\}_{i \in \mathcal{J}}$. It does not know the response lengths $\{o_i\}_{i \in \mathcal{J}}$ in advance. The value of response length o_i is revealed only when job i completes. The benchmark scheduler is clairvoyant and knows all response lengths in advance.

The objective is to minimize *total completion time*, i.e., the sum of the completion times of all jobs. We evaluate non-clairvoyant algorithms against the optimal clairvoyant schedule, which knows all response lengths in advance. Let $\text{OPT}(\mathcal{J})$ denote the minimum total completion time achievable by such a clairvoyant schedule⁶ on the full instance $\mathcal{J}$.⁷ For an algorithm ALG , let $\text{ALG}(\mathcal{J})$ denote its total completion time on $\mathcal{J}$ starting from time 0. We say that ALG is Γ -competitive for $\Gamma \geq 1$ if, for every feasible instance,

$$\text{ALG}(\mathcal{J}) \leq \Gamma \cdot \text{OPT}(\mathcal{J}).$$

For the analysis, we extend these definitions to any subset $\mathcal{J}' \subseteq \mathcal{J}$. Let $\text{OPT}(\mathcal{J}')$ denote the minimum total completion time achievable by a clairvoyant schedule that processes exactly the jobs in $\mathcal{J}'$, and let $\text{ALG}(\mathcal{J}')$ denote the total completion time of ALG on $\mathcal{J}'$ starting from time 0.

An important concept in our algorithms and analysis is the *memory-time area*, which we abbreviate simply as *area* when the context is clear.

⁶We allow the optimal clairvoyant scheduler to preempt or pause jobs (i.e., retain a job in memory without processing it). However, neither is necessary in an optimal solution: delaying a job's start time weakly dominates pausing, and preemption provides no advantage.

⁷With slight abuse of notation, we also use $\mathcal{J}$ to denote both the set of jobs and the corresponding instance.

Definition 1 (Memory-time area). *For a job with prompt length s and response length o , its memory-time area is the cumulative KV-cache memory occupied over a successful run:*

$$A(s, o) \triangleq s \cdot o + \frac{o \cdot (o + 1)}{2}.$$

For job i , we write $A_i \triangleq A(s_i, o_i)$ for the total KV-cache resource consumed by one uninterrupted successful execution of job i .

In addition to its role in our algorithms, the memory-time area provides a clear lower bound on the optimal total completion time. The intuition is that the system can supply at most M units of memory-time area per round, so the aggregate area of all jobs must be paid for in time; placing smaller-area jobs earlier minimizes the resulting sum of completion times.

Lemma 1 (Area-order lower bound). *For every $\mathcal{J}' \subseteq \mathcal{J}$, if we order the jobs as $\pi(1), \dots, \pi(|\mathcal{J}'|)$ so that $A_{\pi(1)} \leq \dots \leq A_{\pi(|\mathcal{J}'|)}$, then*

$$\text{OPT}(\mathcal{J}') \geq \frac{1}{M} \sum_{r=1}^{|\mathcal{J}'|} (|\mathcal{J}'| - r + 1) A_{\pi(r)}.$$

Proof. Consider any clairvoyant feasible schedule for $\mathcal{J}'$ and let $\sigma(1), \dots, \sigma(|\mathcal{J}'|)$ be the jobs ordered by completion time in that schedule. By the time job $\sigma(r)$ completes, the schedule must have supplied the successful memory-time area of the first r completed jobs. Since the system supplies at most M units of memory-time area per round,

$$C_{\sigma(r)} \geq \frac{1}{M} \sum_{q=1}^r A_{\sigma(q)}.$$

Summing over r and exchanging the order of summation gives

$$\sum_{r=1}^{|\mathcal{J}'|} C_{\sigma(r)} \geq \frac{1}{M} \sum_{q=1}^{|\mathcal{J}'|} (|\mathcal{J}'| - q + 1) A_{\sigma(q)}.$$

The right-hand side is minimized when the areas are ordered nondecreasingly, i.e., when $\sigma = \pi$, which yields the claimed bound. $\square$

We also use the following two standard lower bounds.

Lemma 2 (Deletion bound). *If C_i^* is the completion time of job i in an optimal clairvoyant schedule for the full instance, then for any disjoint subsets $\mathcal{J}_1, \dots, \mathcal{J}_k \subseteq \mathcal{J}$,*

$$\sum_{\ell=1}^k \text{OPT}(\mathcal{J}_\ell) \leq \sum_{\ell=1}^k \sum_{i \in \mathcal{J}_\ell} C_i^* \leq \text{OPT}(\mathcal{J}).$$

Proof. Restricting the optimal full-instance schedule to each subset $\mathcal{J}_\ell$ yields a feasible schedule for $\mathcal{J}_\ell$ with total completion time at most $\sum_{i \in \mathcal{J}_\ell} C_i^*$, so $\text{OPT}(\mathcal{J}_\ell) \leq \sum_{i \in \mathcal{J}_\ell} C_i^*$ for each ℓ ; summing over ℓ gives the first inequality. The second inequality holds because $\sum_{i \in \mathcal{J}} C_i^* = \text{OPT}(\mathcal{J})$ by definition. $\square$

Lemma 3 (Processing lower bound). *For every $\mathcal{J}' \subseteq \mathcal{J}$,*

$$\text{OPT}(\mathcal{J}') \geq \sum_{i \in \mathcal{J}'} o_i.$$

Proof. Each job $i \in \mathcal{J}'$ must receive o_i units of processing and can receive at most one unit per round, so it cannot complete before time o_i ; summing over all jobs in $\mathcal{J}'$ gives the claimed bound. $\square$

3 From Area Order to Three Scheduling Cases

In the previous section we derived the area-order lower bound, which gives a tractable benchmark for the optimal clairvoyant completion time. A natural next step is to ask whether a simple scheduling policy can simultaneously achieve high memory utilization and a completion order consistent with this lower bound. In this section we show that, outside the large-memory regime, no single priority rule can satisfy both requirements. This structural impossibility motivates our main algorithmic idea: a decomposition of the job set into three geometric regimes that are handled by specialized sub-schedulers.

The area lower bound in Section 2 suggests that two properties are sufficient for a constant approximation in the clairvoyant setting:

- **(High memory utilization)** the schedule should keep a constant fraction of the memory occupied; and
- **(Area-order completion)** the completion order should be close to nondecreasing memory-time area order.

High memory utilization ensures that the system delivers $\Omega(M)$ units of memory-time area in every round, so the total area of all jobs is paid for within a constant factor of the time needed in the optimal schedule. Area-order completion ensures that the schedule respects the ordering that minimizes the weighted sum in the area lower bound (Lemma 1). Together, they guarantee that the total completion time is bounded by a constant multiple of the lower bound, yielding a constant approximation.

In the clairvoyant large-memory setting where each job satisfies $s_i + o_i = o(M)$, these two properties can be achieved by area-order greedy, which gives a $3 + o(1)$ approximation; see Corollary 2 in Section A.1.

The failure of a single priority rule. Outside the large-memory setting, a single global priority order cannot simultaneously preserve an area-consistent completion order and high memory utilization.

Proposition 1. *No clairvoyant algorithm that follows a single priority rule achieves a constant approximation. An algorithm follows a single priority rule if it fixes a priority order $\prec$ on job types (s, o) and never runs a lower-priority job while an unfinished higher-priority job is feasible.*

Proof. Fix a large integer L and set $M = L^3 + 1$. Consider two job types

$$X = (1, L), \quad Y = (L^3, 1).$$

Type X has area $\Theta(L^2)$ and type Y has area $\Theta(L^3)$. Since a type- Y job occupies $L^3 + 1 = M$ memory in its only active round, it cannot share any round with another job. Below we consider two possible cases of the priority rule.

Case 1: $X \prec Y$. In this case, take one type- X job and $K = \lfloor \sqrt{L} \rfloor$ type- Y jobs. The completion order follows nondecreasing area, but memory utilization is at most $(L + 1)/M = o(1)$ while the type- X job runs alone for L rounds, blocking all type- Y jobs. Thus $\text{ALG} = \Omega(KL) = \Omega(L^{3/2})$. Running the type- Y jobs first gives $\text{OPT} = O(K^2 + L) = O(L)$, hence a ratio of $\Omega(\sqrt{L})$.

Case 2: $Y \prec X$. In this case, take $N = M/(L + 1) = L^2 - L + 1$ type- X jobs and $K = \lfloor L^{3/2} \rfloor$ type- Y jobs. Memory is fully utilized during type- Y rounds, but the completion order reverses the area ranking: all N small-area type- X jobs are blocked until the K serial type- Y jobs complete. Thus $\text{ALG} \geq NK = \Omega(L^{7/2})$. Running all type- X jobs together first (they fit since $N(L + 1) \leq M$) and then the type- Y jobs serially gives $\text{OPT} = O(L^3)$, hence a ratio of $\Omega(\sqrt{L})$.

Combining the two cases together, since $L = \Theta(M^{1/3})$ is unbounded, no such single priority rule gives a constant approximation. $\square$

The proposition rules out any single priority order, including natural global orders such as response-length order and area order. In the non-clairvoyant setting, the obstruction is further compounded because response lengths are unknown and a killed attempt loses all accumulated progress. The two cases impose opposite pressures on a non-clairvoyant scheduler. On one hand, small-prompt jobs must sometimes yield memory to large-prompt jobs; otherwise short large-prompt jobs may remain undiscovered. On the other hand, large-prompt jobs cannot be run too aggressively, because one such attempt may already fill the cache and block many small-prompt jobs. The fundamental obstacle is serving both geometries without letting either monopolize the memory budget. This points to a decomposition into geometric regimes: we first ask whether each regime, taken independently, admits a constant-factor guarantee.⁸

Prompt-heavy jobs. For jobs whose prompt length dominates the response length, the growing memory use is close to a fixed width. If $o_i \leq s_i$, then during a successful attempt,

$$s_i + 1 \leq s_i + u_{i,t} + 1 \leq 2s_i.$$

Hence the dynamic KV-cache memory use is within a constant factor of a fixed rectangle width. This motivates the rectangle strip abstraction: job i is represented by a fixed rectangle width w_i , and the scheduler packs active rectangle widths under the same memory budget.

The fixed-width view gives good utilization when rectangle widths are uniformly small or uniformly large.

1. Suppose every relevant rectangle width is at most Δ . Then utilization comes from greedy packing: if a waiting job cannot be inserted, then the active rectangles already use more than $M - \Delta$ memory. Therefore the schedule has high utilization whenever $M - \Delta = \Omega(M)$.
2. Suppose every relevant rectangle width is at least δ . Then any non-idle round uses at least δ memory. Therefore any work-conserving schedule has constant utilization whenever $\delta = \Omega(M)$.

⁸The hard instance in Proposition 1 mixes large prompt-heavy jobs with response-heavy jobs. In the two cases of the proposition, the failure of the constant ratio comes from violating high-memory utilization and area-order completion, respectively.

Mixing these widths in one schedule can destroy the utilization lower bound. We therefore separate jobs by the prompt length first, using a prompt cutoff $\Lambda \in (0, M/2)$: *large* jobs with $s_i > \Lambda$ are separated first, and the other *small* jobs have bounded rectangle widths under the fixed-width view when $o_i \leq s_i$.

Response-heavy jobs. When $o_i > s_i$ instead, the memory footprint grows well beyond the prompt length and the fixed-width view no longer applies. For these jobs, the area is controlled by the response length:

$$\frac{o_i^2}{2} \leq A_i \leq 2o_i^2.$$

Thus response-length order and area order agree up to constant factors.

Three scheduling cases. The above discussions give the following three regimes:

- **(Large jobs)** $s_i > \Lambda$. These jobs are so wide that each one nearly fills the memory budget alone.
- **(Small prompt-heavy jobs)** $s_i \leq \Lambda$ and $o_i \leq s_i$. The response is no longer than the prompt, so the memory footprint stays within a constant factor of the prompt length.
- **(Small response-heavy jobs)** $s_i \leq \Lambda$ and $o_i > s_i$. The memory footprint grows well beyond the prompt, and the area is dominated by the response length.

Each regime, taken in isolation, admits a constant-competitive non-clairvoyant algorithm. Specifically, Sections 4 and 5 give constant-competitive algorithms for these regimes (Propositions 2 and 3 and Theorem 3). The remaining challenge is that the non-clairvoyant scheduler does not know which regime a job belongs to, because the distinction between prompt-heavy and response-heavy depends on the unknown response length. We explain how to resolve this difficulty when we discuss the meta-scheduler below.

If the three-way partition were known in advance, one could simply run each sub-scheduler on a dedicated replica (a three-fold resource augmentation) and obtain an $O(1)$ -competitive guarantee immediately. Without resource augmentation, the black-box meta-scheduler of Section 6 time-shares a single memory budget across the three sub-schedulers, losing only a constant factor. In the non-clairvoyant setting, the additional difficulty is that the partition is not known: the scheduler cannot distinguish small prompt-heavy jobs from small response-heavy jobs without executing them. We resolve this by routing jobs dynamically as their response lengths reveal themselves, using the routing meta-scheduler described in Section 6.

4 Non-Clairvoyant Rectangle Strip Scheduling

This section analyzes two of the three branches: large branch and prompt-heavy branch. These two branches share the following useful property: for each job, its memory growth can be bounded by a constant factor of its prompt length. In particular, in large branch, the rectangle width is at most M , which is $O(s_i)$ when the prompt cutoff Λ satisfies $\Lambda = \Theta(M)$. In prompt-heavy branch, the width is at most $2s_i$.

This property allows us to reduce the analysis of these two branches to a non-clairvoyant rectangle strip scheduling problem. In this problem, each job is represented by a rectangle with a known fixed width and an unknown processing time. The clairvoyant counterpart of this problem

has been studied in [Cohen et al. \[2024\]](#). To the best of our knowledge, the non-clairvoyant version has not been investigated before. We prove an $O(1)$ -competitive algorithm for this problem, which may also be of independent interest. Next, we define the non-clairvoyant rectangle strip scheduling problem.

4.1 The Rectangle Strip Scheduling Problem

An instance of the rectangle strip problem consists of a job set $\mathcal{J}$, a memory budget M , a known width $w_i \leq M$, and an unknown processing time p_i for each job $i \in \mathcal{J}$. At every round, the total width of the active attempts must be at most M . An active attempt of job i uses width w_i and completes the job after receiving p_i consecutive processing rounds. If an attempt is stopped earlier, its progress is lost. The processing time p_i is not known to the scheduler. Let $\text{OPT}_{\text{rect}}(\mathcal{J})$ denote the optimal clairvoyant total completion time for this rectangle strip instance.

4.2 The Rectangle Strip Scheduler and Analysis

The scheduler RECT processes each job through a sequence of capped attempts with geometrically increasing length caps. Since p_i is unknown, this prevents the schedule from spending too much time on any single job. If the job does not complete within this cap, the attempt expires and is killed. The next attempt restarts the job from zero progress with the doubled cap. Thus the level- r attempt of job i can run for at most 2^r rounds.

Based on this doubling rule, RECT is a greedy allocator over attempts. For attempt (i, r) , define its area budget by

$$B_{i,r} \triangleq w_i 2^r.$$

This is a doubling estimate of the true rectangle area $A_i^{\text{RECT}} \triangleq w_i p_i$. A pending queue $\mathcal{Q}$ is maintained, containing all job attempts waiting to be started. At every event time, the scheduler checks whether the pending attempt with the minimum area budget ($\min \mathcal{Q}$) can fit into the remaining memory. If so, it starts the attempt and repeats the scan; otherwise, the scan stops.

The critical events are time 0, a job completion, and an attempt expiration. Without loss of generality, simultaneous events in Algorithm 1 are processed one at a time under an arbitrary fixed tie-breaking order: each event time after time 0 contains only one completion or one expiration. The pseudocode is given in Algorithm 1.

Proof overview. In the analysis, we decompose the completion time of each job into its active time and queue waiting time. The active time is immediate: the total active time of job i over all its attempts is at most $4p_i$; it remains to bound the queue waiting time. To relate the waiting time to the optimal solution, we use a memory-area argument, extending the clairvoyant argument from Section 3. This argument relies on the following two properties maintained by Algorithm 1:

- (i) attempts are processed in order of nearly nondecreasing rectangle area budget; and
- (ii) every round with a nonempty queue uses a constant fraction of the memory.

Property (ii) turns queue waiting time into an area charge: whenever an attempt waits, the greedy scan and the width conditions ensure that a constant fraction of the memory is occupied by active attempts. Property (i) makes this charge compatible with the area order: the active area charged to a waiting attempt must come only from attempts with no larger budget. This condition lets us bound the total waiting contribution by the area-order lower bound.

Algorithm 1: Budget-Doubling Greedy for Rectangle Strips (RECT)

Input: Rectangle job set $\mathcal{J}$, widths $\{w_i\}_{i \in \mathcal{J}}$, memory M .

```
1  $\mathcal{Q} \leftarrow \{(i, 0) : i \in \mathcal{J}\};$  // attempt  $(i, r)$ : length cap  $2^r$ , budget  $B_{i,r} = w_i 2^r$ 
2 for each event in order: time 0, a completion, or an expiration do
3   if the current event is the completion of an active attempt then
4     | Finalize that completed attempt;
5   else if the current event is the expiration of an active attempt  $(i, r)$  then
6     | Kill  $(i, r)$  and insert  $(i, r + 1)$  into  $\mathcal{Q}$ ;
    // Greedy admission scan until no more jobs can be accepted.
7   while  $\mathcal{Q} \neq \emptyset$  do
8     |  $(i, r) \leftarrow \min \mathcal{Q};$  // the minimum budget attempt
9     | if  $(i, r)$  does not fit in the remaining memory then
10      | break;
11    | Remove  $(i, r)$  from  $\mathcal{Q}$  and start it;
```

The difficulty in proving property (i) is that attempts are created over time. Although the greedy scan always starts a minimum-budget pending attempt when it fits, a replacement attempt (i, r) appears only when $(i, r - 1)$ expires. While $(i, r - 1)$ is active, the scan may have admitted other attempts whose budgets are larger than the budget of the replacement. If (i, r) remained pending, those active attempts would have a larger budget than a pending attempt, violating the desired order.

We rule out this situation using a crucial property of the rectangle strip model: the width of each job is fixed across all attempts, even though the length cap 2^r increases with the level r . When $(i, r - 1)$ expires, it releases a block of memory of width w_i . The replacement (i, r) has the same width w_i , so this released memory is enough to run the replacement. If the replacement budget is small enough to threaten the budget order, then it has priority over every older pending attempt. The following greedy scan therefore starts the replacement immediately, before any older pending attempt can use the released memory. The approximate budget order is formalized by queue monotonicity (Lemma 4): every pending attempt has budget at least that of every active attempt in each settled state.

This fixed-width argument is specific to the rectangle strip regime, which captures both large branch and prompt-heavy branch. In contrast, in the response-heavy regime, the effective width of a job may continue to grow, and therefore the reduction to fixed-width rectangle scheduling no longer applies. For this reason, Section 5 uses a different approach to maintain a good competitive ratio. This separation explains why the branches require different scheduling policies.

To precisely state the property of our algorithm, we index completion and expiration events by $\ell = 1, 2, \dots$, and let $\ell = 0$ denote the initial event at time 0. We call a state *settled* when the greedy admission scan has stopped. Let S_ℓ denote the settled state at the end of the outer **for** loop in Algorithm 1 after event ℓ , and let $\mathcal{Q}(S_\ell)$ denote the pending queue in S_ℓ .

Lemma 4 (Queue monotonicity). *Consider the minimum budget among all pending attempts at a settled state:*

$$\min_{(i,r) \in \mathcal{Q}(S_\ell)} B_{i,r}.$$

This quantity is nondecreasing with $\ell = 0, 1, 2, \dots$, with the convention that its value is $+\infty$ when $\mathcal{Q}(S_\ell) = \emptyset$. Moreover, at any settled state S_ℓ , every active attempt (i, r) and every pending attempt (j, q) satisfy

$$B_{i,r} \leq B_{j,q}.$$

Proof. We prove the two claims by induction on ℓ .

For the base case $\ell = 0$, all attempts initially have level 0 and are placed in the pending queue. The greedy admission scan repeatedly starts a minimum-budget pending attempt until the queue is empty or the minimum-budget pending attempt no longer fits. Thus, at the settled state S_0 , every active attempt has budget at most every remaining pending attempt.

Now fix $\ell \geq 1$, assume both claims hold at settled state $S_{\ell-1}$, and consider event ℓ . We first show that the minimum pending budget does not decrease, by distinguishing two kinds of events.

Completion. Suppose event ℓ is a completion. No new attempt is inserted before the greedy admission scan. The scan only removes attempts from the pending queue and starts them. Thus the minimum pending budget cannot decrease.

Expiration. Suppose event ℓ is the expiration of an active attempt (i, r) . The scheduler kills this attempt and inserts its replacement $(i, r + 1)$. The replacement has the same width w_i and a larger budget.

If

$$B_{i,r+1} \geq \min_{(h,q) \in \mathcal{Q}(S_{\ell-1})} B_{h,q},$$

then inserting $(i, r + 1)$ does not decrease the minimum pending budget. The following greedy admission scan only removes pending attempts and starts them, so the minimum pending budget at S_ℓ is at least the minimum pending budget at $S_{\ell-1}$.

It remains to consider the case

$$B_{i,r+1} < \min_{(h,q) \in \mathcal{Q}(S_{\ell-1})} B_{h,q}.$$

Then $(i, r + 1)$ has strictly smaller budget than every older pending attempt. The expired attempt (i, r) has just released width w_i , and the replacement $(i, r + 1)$ has the same width. Therefore $(i, r + 1)$ fits immediately after the expiration. Since it is the minimum-budget pending attempt, the greedy admission scan starts it before any older pending attempt. Thus $(i, r + 1)$ cannot remain pending in S_ℓ . All attempts that remain pending in S_ℓ are older pending attempts from $\mathcal{Q}(S_{\ell-1})$, except that some of them may have been removed by the scan. Therefore the minimum pending budget at S_ℓ is again at least the minimum pending budget at $S_{\ell-1}$.

Budget order. In both cases, the minimum pending budget does not decrease from $S_{\ell-1}$ to S_ℓ . An active attempt in S_ℓ either survived from $S_{\ell-1}$ or was started during the greedy admission scan after event ℓ . If it survived from $S_{\ell-1}$, the induction hypothesis bounds its budget by the minimum pending budget at $S_{\ell-1}$; since the minimum pending budget does not decrease, its budget is also at most every pending budget at S_ℓ . If it was started during the scan, then it was a minimum-budget pending attempt at the moment it was started. Because the scan always removes a current minimum-budget pending attempt, any attempt started by the scan has budget at most every attempt that remains pending when the scan stops. Thus every active attempt in S_ℓ has budget at most every pending attempt in S_ℓ . $\square$

Write $A_i^{\text{RECT}} \triangleq w_i p_i$ for the rectangle area of job i . Let $r_i \triangleq \min\{r : 2^r \geq p_i\}$ be the first successful level. Then

$$A_i^{\text{RECT}} \leq B_{i,r_i} < 2A_i^{\text{RECT}}.$$

Lemma 5 (Per-job completion-time control). *Assume the rectangle instance on $\mathcal{J}$ has waiting utilization U : in every settled state where $\mathcal{Q} \neq \emptyset$, the active attempts use at least U total memory. For every job $i \in \mathcal{J}$,*

$$C_i^{\text{RECT}} \leq 4p_i + \frac{4}{U} \sum_{j \in \mathcal{J}} \min\{A_i^{\text{RECT}}, A_j^{\text{RECT}}\}.$$

Proof. Let W_i be the total time during which there exists an attempt of job i pending before it starts. The active time is bounded by the geometric caps:

$$\sum_{r=0}^{r_i} 2^r < 2^{r_i+1} \leq 4p_i.$$

Hence

$$C_i^{\text{RECT}} \leq 4p_i + W_i.$$

Consider any waiting round counted by W_i . Let (i, q) be the pending attempt of job i in that round. By Lemma 4, every active attempt (j, r) satisfies

$$B_{j,r} \leq B_{i,q} \leq B_{i,r_i}.$$

The utilization assumption gives at least U active memory in this round, all from attempts of budget at most B_{i,r_i} . Therefore

$$UW_i \leq \sum_{j \in \mathcal{J}} \sum_{\substack{0 \leq q \leq r_j \\ B_{j,q} \leq B_{i,r_i}}} B_{j,q}.$$

For each fixed job j ,

$$\sum_{\substack{0 \leq q \leq r_j \\ B_{j,q} \leq B_{i,r_i}}} B_{j,q} < 2 \min\{B_{i,r_i}, B_{j,r_j}\} \leq 4 \min\{A_i^{\text{RECT}}, A_j^{\text{RECT}}\},$$

where the last inequality uses the successful budget bound above. Combining the previous displays gives

$$W_i \leq \frac{4}{U} \sum_{j \in \mathcal{J}} \min\{A_i^{\text{RECT}}, A_j^{\text{RECT}}\}.$$

Together with the active-time bound above, this proves the lemma. $\square$

Lemma 6 (Rectangle lower bounds). *For every rectangle strip instance on $\mathcal{J}$,*

$$\text{OPT}_{\text{rect}}(\mathcal{J}) \geq \sum_{i \in \mathcal{J}} p_i$$

and

$$\frac{1}{2M} \sum_{i,j \in \mathcal{J}} \min\{A_i^{\text{RECT}}, A_j^{\text{RECT}}\} \leq \text{OPT}_{\text{rect}}(\mathcal{J}).$$

Proof. The processing lower bound is immediate. For the area lower bound, order the jobs so that

$$A_{\pi(1)}^{\text{RECT}} \leq A_{\pi(2)}^{\text{RECT}} \leq \dots \leq A_{\pi(|\mathcal{J}|)}^{\text{RECT}}.$$

The same area-order lower-bound argument (Lemma 1) applied to rectangle areas gives

$$\text{OPT}_{\text{rect}}(\mathcal{J}) \geq \frac{1}{M} \sum_{r=1}^{|\mathcal{J}|} (|\mathcal{J}| - r + 1) A_{\pi(r)}^{\text{RECT}}.$$

Also,

$$\sum_{i,j \in \mathcal{J}} \min\{A_i^{\text{RECT}}, A_j^{\text{RECT}}\} = \sum_{r=1}^{|\mathcal{J}|} (2(|\mathcal{J}| - r) + 1) A_{\pi(r)}^{\text{RECT}} \leq 2 \sum_{r=1}^{|\mathcal{J}|} (|\mathcal{J}| - r + 1) A_{\pi(r)}^{\text{RECT}}.$$

The claim follows. $\square$

Lemma 7 (Width utilization). *Suppose every job satisfies*

$$\delta \leq w_i \leq \Delta.$$

Then the rectangle instance has waiting utilization

$$U = \max\{\delta, M - \Delta\}.$$

Proof. Consider a settled state in which an attempt is waiting. Let m be the total active memory. Let $(i, r) = \min \mathcal{Q}$. Since (i, r) does not fit and has width at most Δ , we have

$$m > M - \Delta.$$

Also, the active set is nonempty; otherwise (i, r) would fit because $w_i \leq M$. Since every active attempt has width at least δ , we also have

$$m \geq \delta.$$

Thus the instance has waiting utilization $\max\{\delta, M - \Delta\}$. $\square$

Theorem 2 (Rectangle strip guarantee). *Suppose every job satisfies*

$$\delta \leq w_i \leq \Delta.$$

Then

$$\text{RECT}(\mathcal{J}) \leq \left(4 + \frac{8M}{\max\{\delta, M - \Delta\}}\right) \text{OPT}_{\text{rect}}(\mathcal{J}).$$

Proof. Let $U = \max\{\delta, M - \Delta\}$. By Lemma 7, the rectangle instance has waiting utilization U . Summing Lemma 5 over all jobs gives

$$\text{RECT}(\mathcal{J}) \leq 4 \sum_{i \in \mathcal{J}} p_i + \frac{4}{U} \sum_{i,j \in \mathcal{J}} \min\{A_i^{\text{RECT}}, A_j^{\text{RECT}}\}.$$

The theorem follows from Lemma 6. $\square$

Remark 1. *For a general rectangle strip instance with no width range assumption, the same scheduler can be applied to two width classes and combined by a black-box doubling meta-scheduler defined in Section 6.1. This gives a constant-competitive algorithm for the full rectangle strip problem; the complete statement and proof are in Section B.*

4.3 Applying to Homogeneous KV-Cache Regimes

The large branch and the prompt-heavy branch are both applications of RECT to homogeneous KV-cache regimes. When every job satisfies the same regime condition, the corresponding branch is an ordinary rectangle strip schedule with a direct completion-time guarantee. They are defined as follows.

- **Large branch.** Use RECT with rectangle width $w_i = M$ and processing time $p_i = o_i$.
- **Prompt-heavy branch.** Use RECT with rectangle width $w_i = 2s_i$ and processing time $p_i = o_i$.

Proposition 2 (Large homogeneous guarantee). *If every job satisfies $s_i > \Lambda$, then the large branch schedule has total completion time at most*

$$\left(4 + \frac{8M}{\Lambda}\right) \text{OPT}(\mathcal{J}).$$

Proof. By per-job feasibility, every rectangle completion is true completion in the original KV-cache instance. Taking $\delta = \Delta = M$ satisfies the width assumptions in Theorem 2. Also, for every job i ,

$$A_i^{\text{RECT}} = Mo_i \leq \frac{M}{\Lambda} s_i o_i \leq \frac{M}{\Lambda} A_i.$$

As in the proof of Theorem 2, by Lemma 5 and Lemma 7,

$$\text{RECT}(\mathcal{J}) \leq 4 \sum_{i \in \mathcal{J}} o_i + \frac{4}{M} \sum_{i,j \in \mathcal{J}} \min\{A_i^{\text{RECT}}, A_j^{\text{RECT}}\} \leq 4 \sum_{i \in \mathcal{J}} o_i + \frac{4}{M} \frac{M}{\Lambda} \sum_{i,j \in \mathcal{J}} \min\{A_i, A_j\}$$

As in the proof of Lemma 6, by the area-order lower bound argument in Lemma 1,

$$\sum_{i \in \mathcal{J}} o_i \leq \text{OPT}(\mathcal{J}),$$

and

$$\frac{1}{2M} \sum_{i,j \in \mathcal{J}} \min\{A_i, A_j\} \leq \text{OPT}(\mathcal{J}).$$

Thus we conclude

$$\text{RECT}(\mathcal{J}) \leq \left(4 + \frac{8M}{\Lambda}\right) \text{OPT}(\mathcal{J}).$$

□

Proposition 3 (Prompt-heavy homogeneous guarantee). *If every job satisfies $s_i \leq \Lambda$ and $o_i \leq s_i$, then the prompt-heavy branch schedule has total completion time at most*

$$\left(4 + \frac{16M}{M - 2\Lambda}\right) \text{OPT}(\mathcal{J}).$$

Proof. Under the prompt-heavy condition, every rectangle completion is true completion in the original KV-cache instance. Every rectangle width is at most 2Λ . Taking $\delta = 0$ and $\Delta = 2\Lambda$ satisfies the width assumptions in Theorem 2. Moreover,

$$A_i^{\text{RECT}} = 2s_i o_i \leq 2A_i.$$

The same argument as Large branch (Proposition 2) gives

$$\begin{aligned} \text{RECT}(\mathcal{J}) &\leq 4 \sum_{i \in \mathcal{J}} o_i + \frac{4}{M - 2\Lambda} \sum_{i,j \in \mathcal{J}} \min\{A_i^{\text{RECT}}, A_j^{\text{RECT}}\} \\ &\leq 4 \sum_{i \in \mathcal{J}} o_i + \frac{8}{M - 2\Lambda} \sum_{i,j \in \mathcal{J}} \min\{A_i, A_j\} \\ &\leq \left(4 + \frac{16M}{M - 2\Lambda}\right) \text{OPT}(\mathcal{J}). \end{aligned}$$

□

Remark 2. For $\Lambda = M/4$, the constants in Propositions 2 and 3 are both equal to 36.

5 Area-Geometric Slicing for Response-heavy Jobs

Response-heavy jobs create a different obstacle from the rectangle strip setting of Section 4. Their memory footprint can grow far beyond the prompt length, so the fixed-width queue-monotonicity argument from Section 4 no longer applies.

This section uses a structural fact absent from the prompt-heavy regime: response length determines area up to constant factors. Every job i considered in this section satisfies

$$s_i < o_i, \quad s_i + o_i \leq M.$$

Under this condition, memory-time area and the squared response length are equivalent up to constants:

$$\frac{o_i^2}{2} \leq A_i \leq 2o_i^2.$$

Thus ordering jobs by response caps also orders their area scales within constant factors. This equivalence permits each phase to replace the true prompts with a common prompt upper bound with only a constant-factor loss.

The scheduler RESPONSE uses geometric slicing to exploit this equivalence. The geometric slicing approach of Feng et al. [2026] was developed for the identical-prompt setting. Although prompts may differ across jobs here, the area equivalence above allows each phase of RESPONSE to impose a common response cap and replace heterogeneous prompts with a common proxy prompt. The resulting proxy jobs are scheduled by the SPS subroutine described below. This construction realizes the two sufficient conditions from Section 3: SPS provides high memory utilization within each phase, and the geometric response caps ensure that the completion order is consistent with nearly nondecreasing area.

The SPS subroutine. The Staggered Pipeline Schedule (SPS) is a deterministic subroutine from Feng et al. [2026] for scheduling an ordered batch of identical jobs. An SPS instance consists

of $m \geq 1$ identical jobs, each with common prompt length $s \geq 0$ and slice length $\tau \geq 1$. Given a degree of parallelism $k \geq 1$, the schedule assigns the job of rank $j \in \{0, \dots, m-1\}$ the start offset

$$S_j = \lfloor \frac{j\tau}{k} \rfloor,$$

and runs that job for τ rounds. Consecutive jobs are therefore spaced roughly τ/k rounds apart, so at most k jobs overlap at any time. The key property is efficiency: by staggering the start times, SPS can choose a nearly optimal parallelism.

Fact 1 (SPS properties from [Feng et al. \[2026\]](#)). *For a degree of parallelism k , slice length τ , and common prompt length s , the peak memory of the infinite SPS schedule is*

$$\text{Peak}(k, \tau, s) \triangleq sk + \frac{\tau k + \tau + k - \gcd(\tau, k)}{2}.$$

Hence, if $\text{Peak}(k, \tau, s) \leq M$, then every finite SPS prefix is feasible. In particular, whenever $\tau + s \leq M$, the maximum feasible degree of parallelism satisfies

$$k^*(\tau, s) \triangleq \max\{k \in \mathbb{N}^+ : \text{Peak}(k, \tau, s) \leq M\} \geq \left\lfloor \frac{2M - \tau + 1}{2s + \tau + 1} \right\rfloor \geq 1.$$

The RESPONSE scheduler (Algorithm 2) uses geometrically increasing response caps. Fix a scaling factor $\alpha > 1$. Let $L \triangleq \lceil \log_\alpha M \rceil$. In phase r , the response cap is

$$\tau_r = \min\{\lceil \alpha^r \rceil, M\}.$$

For each phase r , define the proxy prompt

$$\bar{s}_r = \min\{\tau_r, M - \tau_r\}$$

and choose the maximum feasible proxy parallelism $k_r = k^*(\tau_r, \bar{s}_r)$. The SPS call uses the induced order on its current job set from a deterministic order on the input job set; a job that reaches the cap without completing is killed.

Algorithm 2: Response-Geometric SPS (RESPONSE)

Input: Job set $\mathcal{J}$, known prompt lengths $\{s_i\}$, memory M , scaling factor $\alpha > 1$.

```

1  $\mathcal{U} \leftarrow \mathcal{J}$ ;
2 for  $r = 0, 1, \dots, L$  do
3   Set  $\tau_r \leftarrow \min\{\lceil \alpha^r \rceil, M\}$ ;
4   Set  $\bar{s}_r \leftarrow \min\{\tau_r, M - \tau_r\}$ ;
5   Set  $k_r \leftarrow k^*(\tau_r, \bar{s}_r)$ ;
6    $\mathcal{T}_r \leftarrow \{i \in \mathcal{U} : s_i \leq \tau_r\}$ ;
7   Run the SPS \[Feng et al., 2026\] for prompt  $\bar{s}_r$ , slice  $\tau_r$ , and parallelism  $k_r$  on  $\mathcal{T}_r$ ;
   // When the proxy slot of job  $i$  starts, run the real job  $i$ .
   // Kill job  $i$  if it does not complete after  $\tau_r$  rounds.
8   Remove completed jobs from  $\mathcal{U}$ ;
```

Lemma 8 (Feasibility of RESPONSE). *Every phase of Algorithm 2 is feasible.*

Proof. Fix phase r and write $\tau = \tau_r$ and $k = k_r$. Every scheduled job satisfies $s_i \leq \tau$.

If $\tau > M/2$, we have

$$\text{Peak}(2, \tau, M - \tau) = 2(M - \tau) + \frac{2\tau + \tau + 2 - \gcd(\tau, 2)}{2} \geq 2M - \tau + 1 > M,$$

hence $k = 1$; each job runs sequentially, and per-job feasibility $s_i + o_i \leq M$ bounds the memory in every round.

If $\tau \leq M/2$, then $\bar{s}_r = \tau \geq s_i$, so the real memory $s_i + u + 1$ at progress u does not exceed the proxy memory $\bar{s}_r + u + 1$; the proxy SPS schedule is feasible by Fact 1 and the choice of k . $\square$

Lemma 9 (Completion of RESPONSE). *Let r be the first phase with $\tau_r \geq o_i$. Job i completes in phase r .*

Proof. For any phase $q < r$, the cap satisfies $\tau_q < o_i$, so job i receives fewer than o_i rounds and cannot complete. In phase r , response dominance gives $s_i < o_i \leq \tau_r$, so job i is eligible. The phase runs job i for up to τ_r rounds. Since $\tau_r \geq o_i$, the job completes in that phase. $\square$

The following theorem gives the response-heavy guarantee used by the meta-scheduler.

Theorem 3. *For every fixed $\alpha > 1$, the scheduler RESPONSE satisfies*

$$\text{RESPONSE}(\mathcal{J}) = O_\alpha(1) \text{OPT}(\mathcal{J})$$

on every response-heavy instance $\mathcal{J}$. In particular, for $\alpha = 2$,

$$\text{RESPONSE}(\mathcal{J}) \leq \frac{236}{3} \text{OPT}(\mathcal{J}) \approx 78.67 \text{OPT}(\mathcal{J}).$$

Proof overview. Let $\tau_{-1} = 0$, and let $\mathcal{J}_r = \{i \in \mathcal{J} : \tau_{r-1} < o_i \leq \tau_r\}$ and $n_r = |\mathcal{J}_r|$ for $r = 0, 1, \dots, L$. Write $N_{\geq r} \triangleq \sum_{q=r}^L n_q$ for the number of jobs in classes $r, r+1, \dots, L$. By Lemma 9, every job in $\mathcal{J}_r$ completes in phase r . Let D_r denote the duration of phase r . By the start-offset rule of SPS, the last start time in phase r is less than $\tau_r N_{\geq r}/k_r$. Thus

$$D_r \leq \tau_r + \frac{\tau_r}{k_r} N_{\geq r}.$$

The lower bound on k^* implies $k_r = \Omega(M/\tau_r)$ when $\tau_r \leq M/2$, while $k_r = 1$ when $\tau_r > M/2$. The choice of k_r gives $\tau_r/k_r = O(\tau_r^2/M)$, so

$$\text{RESPONSE}(\mathcal{J}) = O \left(\sum_{r=0}^L \tau_r N_{\geq r} + \frac{1}{M} \sum_{r=0}^L \tau_r^2 N_{\geq r}^2 \right).$$

The linear sum is charged to the processing lower bound because the caps grow geometrically. For the quadratic sum, expand the square over ordered pairs of jobs and charge each pair to the smaller response class. Response dominance gives $A_i = \Theta(o_i^2)$, so this pairwise charge is bounded by the area-order lower bound.

The complete proof is deferred to Section C.

6 Meta-Scheduling with Routing

If the cases in Section 3 were known in advance, the scheduler could run the corresponding sub-schedulers through a black-box meta-scheduler on one shared memory resource. This known-partition reduction is given in Section 6.1. The black-box meta-scheduler runs the sub-schedulers in a round-robin order over doubling stages and preserves the total completion-time objective up to a constant factor.

The non-clairvoyant setting, however, does not directly provide this partition. For jobs with $s_i \leq \Lambda$, the distinction between $o_i \leq s_i$ (prompt-heavy) and $o_i > s_i$ (response-heavy) depends on the unknown response length. The meta-scheduler therefore needs to *route* small jobs between sub-schedulers: a small job first enters the prompt-heavy branch; this branch either completes the job or reaches the boundary s_i and certifies $o_i > s_i$. Certified jobs are then routed to the response-heavy branch. The routing meta-scheduler in Section 6.2 uses the same doubling stages as the black-box meta-scheduler to combine these two events and gives a constant-competitive guarantee.

6.1 The Black-Box Meta-Scheduler

The meta-scheduler in Algorithm 3 merges a constant number of job partitions into one schedule and loses only a constant factor in the competitive guarantee. In each fresh invocation of a sub-scheduler, every job that has already completed in the meta-scheduler is still included as a *dummy job*. They follow exactly the schedule produced by the sub-scheduler, but their scheduled processing is ignored. This allows the meta-scheduler to invoke each sub-scheduler as an unchanged black box, without requiring any modification or any structural assumptions about its behavior.

Algorithm 3: Black-Box Meta-Scheduler (BLACKBOXMETA)

Input: Job partition $\mathcal{J}_1, \dots, \mathcal{J}_k$ and corresponding sub-schedulers $\text{ALG}_1, \dots, \text{ALG}_k$.

```

1 for stage  $r = 0, 1, 2, \dots$  do
2   for  $\ell = 1, 2, \dots, k$  do
3     Run a fresh copy of  $\text{ALG}_\ell$  on  $\mathcal{J}_\ell$  for  $2^r$  rounds;
     // Completed jobs remain in the fresh copy as dummy jobs.
     // Their scheduled processing is idle.
```

Lemma 10 (Black-box per-job completion control). *Suppose that $\mathcal{J}$ is partitioned into disjoint sets $\mathcal{J}_1, \dots, \mathcal{J}_k$. For each $\ell = 1, \dots, k$, let C_i^ℓ be the completion time of job $i \in \mathcal{J}_\ell$ in the fresh run $\text{ALG}_\ell(\mathcal{J}_\ell)$. Then the meta-scheduler BLACKBOXMETA in Algorithm 3 completes every job $i \in \mathcal{J}_\ell$ by time*

$$C_i^{\text{BLACKBOXMETA}(\mathcal{J})} \leq (2(k + \ell - 1) + 1)C_i^\ell \leq (4k - 1)C_i^\ell.$$

Proof. For each $i \in \mathcal{J}_\ell$, let r be the first stage with $2^r \geq C_i^\ell$, at which job i completes. The ℓ -th call of stage r starts after at most another $(\ell - 1)2^r$ rounds. Therefore

$$C_i^{\text{BLACKBOXMETA}(\mathcal{J})} \leq k(2^r - 1) + (\ell - 1)2^r + C_i^\ell \leq (2(k + \ell - 1) + 1)C_i^\ell \leq (4k - 1)C_i^\ell.$$

The claim follows. □

Corollary 1 (Black-box competitive guarantee). *Under the assumptions and notation of Lemma 10, suppose that each sub-scheduler ALG_ℓ satisfies*

$$\text{ALG}_\ell(\mathcal{J}_\ell) \leq \Gamma_\ell \cdot \text{OPT}(\mathcal{J}_\ell).$$

Let $\Gamma = \max_{\ell=1,\dots,k} \Gamma_\ell$. Then the meta-scheduler BLACKBOXMETA in Algorithm 3 satisfies

$$\text{BLACKBOXMETA}(\mathcal{J}) \leq (4k - 1)\Gamma \cdot \text{OPT}(\mathcal{J}).$$

Proof. By Lemma 10,

$$\text{BLACKBOXMETA}(\mathcal{J}) \leq (4k - 1) \sum_{\ell=1}^k \text{ALG}_\ell(\mathcal{J}_\ell).$$

The assumed sub-scheduler guarantees give

$$\text{BLACKBOXMETA}(\mathcal{J}) \leq (4k - 1)\Gamma \sum_{\ell=1}^k \text{OPT}(\mathcal{J}_\ell).$$

Since $\mathcal{J}_1, \dots, \mathcal{J}_k$ form a partition of $\mathcal{J}$, $\sum_{\ell=1}^k \text{OPT}(\mathcal{J}_\ell) \leq \text{OPT}(\mathcal{J})$ holds by Lemma 2. $\square$

6.2 The Routing Meta-Scheduler

The black-box reduction assumes that every job is already assigned to the correct sub-scheduler. The non-clairvoyant algorithm does not have this partition: for jobs with $s_i \leq \Lambda$, it must decide online whether a job is prompt-heavy or response-heavy.

The large branch and prompt-heavy branch use the rectangle strip scheduler RECT defined in Section 4. They differ only in how completion in the rectangle strip instance is interpreted in the original KV-cache instance.

- **Large branch $\mathcal{L}$.** Jobs with large prompts are run by the full-width rectangle scheduler; every rectangle completion is true completion.
- **Prompt-heavy branch $\mathcal{P}$.** Jobs in the prompt pool are run by the capped rectangle scheduler. In each run, the current pool $\mathcal{P}$ is represented by the capped instance $\widehat{\mathcal{P}}$, with the original prompt lengths and response lengths capped at $\min\{o_i, s_i\}$. Completion in $\widehat{\mathcal{P}}$ is either true completion in the original instance or certification that $o_i > s_i$; certified jobs move to $\mathcal{R}$.
- **Response-heavy branch $\mathcal{R}$.** Certified response-heavy jobs are run by the response-heavy scheduler RESPONSE .

$\mathcal{L}$ is known in advance; $\mathcal{P}$ is initialized to $\{i \in \mathcal{J} : s_i \leq \Lambda\}$ and $\mathcal{R}$ is initially empty. During the algorithm, certified but unfinished jobs move from $\mathcal{P}$ to $\mathcal{R}$. As in the black-box meta-scheduler, completed jobs remain in their branch pool and are treated as dummy jobs in fresh runs; their scheduled processing is idle.⁹

⁹This is only a notational convention. The same analysis also applies if completed jobs are removed from their branch pools, because each later branch call is on a subset of the corresponding reference instance.

Algorithm 4: Routing Meta-Scheduler (ROUTE)

Input: Job set $\mathcal{J}$, memory M , prompt cutoff Λ , and branch schedulers large branch, prompt-heavy branch, response-heavy branch.

```
1  $\mathcal{L} \leftarrow \{i \in \mathcal{J} : s_i > \Lambda\}$  ; // large branch
2  $\mathcal{P} \leftarrow \{i \in \mathcal{J} : s_i \leq \Lambda\}$  ; // prompt-heavy branch
3  $\mathcal{R} \leftarrow \emptyset$  ; // response-heavy branch
4 for stage  $r = 0, 1, 2, \dots$  do
5   Run a fresh copy of the prompt-heavy branch scheduler on  $\mathcal{P}$  for  $2^r$  rounds;
6   Move every newly certified job from  $\mathcal{P}$  to  $\mathcal{R}$ ;
7   Run a fresh copy of the response-heavy branch scheduler on  $\mathcal{R}$  for  $2^r$  rounds;
8   Run a fresh copy of the large branch scheduler on  $\mathcal{L}$  for  $2^r$  rounds;
```

Analysis ingredients. For the analysis, define

$$\mathcal{S} \triangleq \mathcal{J} \setminus \mathcal{L} = \{i \in \mathcal{J} : s_i \leq \Lambda\}, \quad \mathcal{P}^* \triangleq \{i \in \mathcal{S} : o_i \leq s_i\}, \quad \mathcal{R}^* \triangleq \{i \in \mathcal{S} : o_i > s_i\}.$$

Let $\widehat{\mathcal{S}}$ denote the auxiliary capped instance on $\mathcal{S}$: it has the same jobs and prompt lengths as $\mathcal{S}$, and response length $\min\{o_i, s_i\}$ for each job i . Thus every capped prompt-pool instance $\widehat{\mathcal{P}}$ that appears during the algorithm is a subinstance of $\widehat{\mathcal{S}}$. The instance $\widehat{\mathcal{S}}$ is a relaxation of $\mathcal{S}$: every feasible schedule for $\mathcal{S}$ is feasible for $\widehat{\mathcal{S}}$, so $\text{OPT}(\widehat{\mathcal{S}}) \leq \text{OPT}(\mathcal{S})$.

The branch calls made by the meta-scheduler are controlled by bounds defined on the corresponding full sets.

- For the rectangle branches, let $C_i^{\mathcal{L}}$ ($i \in \mathcal{L}$) and $C_i^{\mathcal{P}}$ ($i \in \mathcal{S}$) be the per-job bounds from Lemma 5 for the large branch instance on $\mathcal{L}$ and the capped instance $\widehat{\mathcal{S}}$, respectively. The sum in Lemma 5 only decreases under job deletion, so these bounds remain valid for every rectangle branch call on a current subset.
- For the response-heavy branch, let $C_i^{\mathcal{R}}$ be the completion time of i in $\text{RESPONSE}(\mathcal{R}^*)$. By Lemma 15, $C_i^{\mathcal{R}}$ also bounds the completion time of i in every response-heavy branch call on a certified subset containing i .

Lemma 11 (Routing loss). *Consider any job $i \in \mathcal{R}^*$. Then Algorithm 4 completes i by time*

$$C_i^{\text{ROUTE}} \leq \max\{8C_i^{\mathcal{P}} + C_i^{\mathcal{R}}, 9C_i^{\mathcal{R}}\}.$$

Proof. Let $r_i^{\mathcal{P}}$ be the first stage with $2^{r_i^{\mathcal{P}}} \geq C_i^{\mathcal{P}}$, and let $r_i^{\mathcal{R}}$ be the first stage with $2^{r_i^{\mathcal{R}}} \geq C_i^{\mathcal{R}}$. By minimality, $2^{r_i^{\mathcal{P}}} < 2C_i^{\mathcal{P}}$ and $2^{r_i^{\mathcal{R}}} < 2C_i^{\mathcal{R}}$.

Each stage q consists of three calls, each of length 2^q . Hence the total time before stage r is

$$3 \sum_{q=0}^{r-1} 2^q = 3(2^r - 1).$$

The prompt-heavy branch call in stage r ends by time $3(2^r - 1) + 2^r$, and the response-heavy branch call in the same stage starts at that time.

If $r_i^P \leq r_i^R$, then job i has been certified by the time the response-heavy branch call of stage r_i^R starts. By Lemma 15, this call completes i within C_i^R additional rounds. Therefore

$$C_i^{\text{ROUTE}} \leq 3(2^{r_i^R} - 1) + 2^{r_i^R} + C_i^R \leq 9C_i^R.$$

If $r_i^P > r_i^R$, then job i is certified during the prompt-heavy branch call of stage r_i^P and is included in the following response-heavy branch call of the same stage. Since $r_i^P > r_i^R$, this response-heavy branch call has length at least C_i^R . Again by Lemma 15,

$$C_i^{\text{ROUTE}} \leq 3(2^{r_i^P} - 1) + 2^{r_i^P} + C_i^R \leq 8C_i^P + C_i^R.$$

Combining the two cases proves the lemma. $\square$

Theorem 1. *For every feasible batch KV-cache scheduling instance $\mathcal{J}$ with arbitrary prompt lengths and arbitrary response lengths, there exists a fully non-clairvoyant scheduling algorithm whose total completion time is within a constant factor of the optimal clairvoyant total completion time:*

$$\text{ALG}(\mathcal{J}) = O(1) \cdot \text{OPT}(\mathcal{J}).$$

Proof. Set $\Lambda = M/4$ and $\alpha = 2$. Run the routing meta-scheduler ROUTE (Algorithm 4). We first bound each job's completion time by the corresponding branch cost, and then compare the branch sums with OPT.

By Lemma 10, applied to the prompt-heavy branch and large branch calls,

$$C_i^{\text{ROUTE}} \leq 7C_i^P \quad (i \in \mathcal{P}^*), \quad C_i^{\text{ROUTE}} \leq 11C_i^L \quad (i \in \mathcal{L}).$$

Together with Lemma 11, this gives

$$\text{ROUTE}(\mathcal{J}) \leq 11 \sum_{i \in \mathcal{L}} C_i^L + 8 \sum_{i \in \mathcal{S}} C_i^P + 9 \sum_{i \in \mathcal{R}^*} C_i^R.$$

With $\Lambda = M/4$, Propositions 2 and 3 give

$$\sum_{i \in \mathcal{L}} C_i^L \leq 36 \text{OPT}(\mathcal{L}), \quad \sum_{i \in \mathcal{S}} C_i^P \leq 36 \text{OPT}(\widehat{\mathcal{S}}) \leq 36 \text{OPT}(\mathcal{S}).$$

For the response branch, Theorem 3 and the deletion bound (Lemma 2) give

$$\sum_{i \in \mathcal{R}^*} C_i^R = \text{RESPONSE}(\mathcal{R}^*) \leq \frac{236}{3} \text{OPT}(\mathcal{R}^*) \leq \frac{236}{3} \text{OPT}(\mathcal{S}).$$

Since $\mathcal{L}$ and $\mathcal{S}$ partition $\mathcal{J}$, combining with the deletion bound (Lemma 2) gives

$$\text{ROUTE}(\mathcal{J}) \leq 396 \text{OPT}(\mathcal{L}) + (288 + 708) \text{OPT}(\mathcal{S}) \leq 996 \text{OPT}.$$

$\square$

Remark 3 (Computational complexity of ROUTE). *Although the scheduling model describes decisions round by round, the total number of discrete events across RECT, RESPONSE, and ROUTE is bounded by $\text{poly}(n, \log M)$. An event-driven implementation that jumps between successive events therefore runs in polynomial time.*

7 Conclusion and Discussion

We have presented the first constant-competitive non-clairvoyant scheduling algorithm for batched LLM inference under a hard KV-cache memory budget with arbitrary prompt lengths and arbitrary response lengths. The core algorithmic idea is a regime-aware routing framework that decomposes jobs into three geometric regimes—large jobs, small prompt-heavy jobs, and small response-heavy jobs—and handles each with a specialized sub-scheduler. A meta-scheduler time-shares the memory budget across these regimes and dynamically routes jobs as their execution reveals their true geometric class. This yields an $O(1)$ -competitive guarantee for total completion time, and the same framework extends to other variants and extensions such as makespan minimization and online arrivals.

From a systems perspective, our results provide a principled foundation for a design principle already present in modern serving stacks: global request routing across replicas. Rather than introducing routing complexity inside a single replica, our framework shows that partitioning jobs by their geometric structure—large jobs, prompt-heavy jobs, and response-heavy jobs—and dispatching each class to a dedicated replica with a simple specialized local scheduler can simultaneously achieve worst-case robustness and practical efficiency. Because the theoretical meta-scheduler must time-share a single memory budget across heterogeneous classes, a multi-replica implementation actually attains a better competitive ratio than our single-replica bound, while remaining relatively simpler to deploy with existing continuous-batching and block-based cache managers. In this sense, simple local rules, properly routed by geometric class, suffice for both theoretical and practical performance.

References

- Anna Adamaszek, Tomasz Kociumaka, Marcin Pilipczuk, and Michal Pilipczuk. Hardness of approximation for strip packing. *ACM Trans. Comput. Theory*, 9(3):14:1–14:7, 2017. doi: 10.1145/3092026. URL <https://doi.org/10.1145/3092026>.
- Helmut Alt, Mark de Berg, and Christian Knauer. Approximating minimum-area rectangular and convex containers for packing convex polygons. *J. Comput. Geom.*, 8(1):1–10, 2017. doi: 10.20382/JOCG.V8I1A1. URL <https://doi.org/10.20382/jocg.v8i1a1>.
- Ruicheng Ao, Gan Luo, David Simchi-Levi, and Xinshang Wang. Optimizing LLM inference: Fluid-guided online scheduling with memory constraints. *CoRR*, abs/2504.11320, 2025. doi: 10.48550/ARXIV.2504.11320. URL <https://doi.org/10.48550/arXiv.2504.11320>.
- Luca Becchetti and Stefano Leonardi. Nonclairvoyant scheduling to minimize the total flow time on single and parallel machines. *J. ACM*, 51(4):517–539, 2004. doi: 10.1145/1008731.1008732. URL <https://doi.org/10.1145/1008731.1008732>.
- Luca Becchetti, Stefano Leonardi, Alberto Marchetti-Spaccamela, and Kirk Pruhs. Semi-clairvoyant scheduling. *Theor. Comput. Sci.*, 324(2-3):325–335, 2004. doi: 10.1016/J.TCS.2004.05.023. URL <https://doi.org/10.1016/j.tcs.2004.05.023>.
- Ziyad Benomar and Vianney Perchet. Non-clairvoyant scheduling with partial predictions. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings*

- of *Machine Learning Research*, pages 3506–3538. PMLR, 2024. URL <https://proceedings.mlr.press/v235/benomar24a.html>.
- Ziyad Benomar, Romain Cosson, Alexander Lindermayr, and Jens Schlöter. Non-clairvoyant scheduling with progress bars. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025*, 2025. URL https://papers.nips.cc/paper_files/paper/2025/hash/868f2266086530b2c71006ea1908b14a-Abstract-Conference.html.
- Chandra Chekuri, Rajeev Motwani, B. Natarajan, and Clifford Stein. Approximation techniques for average completion time scheduling. *SIAM J. Comput.*, 31(1):146–166, 2001. doi: 10.1137/S0097539797327180. URL <https://doi.org/10.1137/S0097539797327180>.
- Qingyun Chen, Sungjin Im, and Aditya Petety. Online scheduling via gradient descent for weighted flow time minimization. In Yossi Azar and Debmalya Panigrahi, editors, *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2025, New Orleans, LA, USA, January 12-15, 2025*, pages 3802–3841. SIAM, 2025a. doi: 10.1137/1.9781611978322.128. URL <https://doi.org/10.1137/1.9781611978322.128>.
- Zixi Chen, Yinyu Ye, and Zijie Zhou. Adaptively robust LLM inference optimization under prediction uncertainty. *CoRR*, abs/2508.14544, 2025b. doi: 10.48550/ARXIV.2508.14544. URL <https://doi.org/10.48550/arXiv.2508.14544>.
- Ilan Reuven Cohen, Izack Cohen, and Iyar Zaks. A theoretical and empirical study of job scheduling in cloud computing environments: The weighted completion time minimization problem with capacitated parallel machines. *Ann. Oper. Res.*, 338(1):429–452, 2024. doi: 10.1007/S10479-023-05613-X. URL <https://doi.org/10.1007/s10479-023-05613-x>.
- J. G. Dai, Tianze Deng, Yueying Li, and Tianyi Peng. Throughput-optimal scheduling algorithms for LLM inference and AI agents. *CoRR*, abs/2504.07347, 2025. doi: 10.48550/ARXIV.2504.07347. URL <https://doi.org/10.48550/arXiv.2504.07347>.
- Leah Epstein and Rob van Stee. Lower bounds for on-line single-machine scheduling. *Theor. Comput. Sci.*, 299(1-3):439–450, 2003. doi: 10.1016/S0304-3975(02)00488-7. URL [https://doi.org/10.1016/S0304-3975\(02\)00488-7](https://doi.org/10.1016/S0304-3975(02)00488-7).
- Yiding Feng, Zonghan Yang, and Yuhao Zhang. Competitive non-clairvoyant kv-cache scheduling for LLM inference. *CoRR*, abs/2601.22996, 2026. doi: 10.48550/ARXIV.2601.22996. URL <https://doi.org/10.48550/arXiv.2601.22996>.
- Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. Efficient LLM scheduling by learning to rank. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/6c8985579293e0209bdaa4f21bb1d237-Abstract-Conference.html.

- Leslie A. Hall, Andreas S. Schulz, David B. Shmoys, and Joel Wein. Scheduling to minimize average completion time: Off-line and on-line approximation algorithms. *Math. Oper. Res.*, 22(3):513–544, 1997. doi: 10.1287/MOOR.22.3.513. URL <https://doi.org/10.1287/moor.22.3.513>.
- Sungjin Im, Janardhan Kulkarni, and Kamesh Munagala. Competitive algorithms from competitive equilibria: Non-clairvoyant scheduling under polyhedral constraints. *J. ACM*, 65(1):3:1–3:33, 2018. doi: 10.1145/3136754. URL <https://doi.org/10.1145/3136754>.
- Sungjin Im, Ravi Kumar, Mahshid Montazer Qaem, and Manish Purohit. Non-clairvoyant scheduling with predictions. *ACM Trans. Parallel Comput.*, 10(4):19:1–19:26, 2023. doi: 10.1145/3593969. URL <https://doi.org/10.1145/3593969>.
- Sven Jäger, Guillaume Sagnol, Daniel Schmidt genannt Waldschmidt, and Philipp Warode. Competitive kill-and-restart and preemptive strategies for non-clairvoyant scheduling. *Math. Program.*, 210(1):457–509, 2025. doi: 10.1007/S10107-024-02118-8. URL <https://doi.org/10.1007/s10107-024-02118-8>.
- Sven Jäger, Alexander Lindermayr, and Nicole Megow. The power of proportional fairness for nonclairvoyant polytope scheduling. *SIAM J. Comput.*, 55(2):247–279, 2026. doi: 10.1137/25M1763974. URL <https://doi.org/10.1137/25M1763974>.
- Patrick Jaillet, Jiashuo Jiang, Konstantina Mellou, Marco Molinaro, Chara Podimata, and Zijie Zhou. Online scheduling for llm inference with kv cache constraints. *arXiv preprint arXiv:2502.07115*, 2025. URL <https://arxiv.org/abs/2502.07115>.
- Klaus Jansen and Malin Rau. Closing the gap for single resource constraint scheduling. In Petra Mutzel, Rasmus Pagh, and Grzegorz Herman, editors, *29th Annual European Symposium on Algorithms, ESA 2021, Lisbon, Portugal (Virtual Conference), September 6-8, 2021*, volume 204 of *LIPIcs*, pages 53:1–53:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021. doi: 10.4230/LIPICS.ESA.2021.53. URL <https://doi.org/10.4230/LIPICS.ESA.2021.53>.
- Klaus Jansen and Guochuan Zhang. Maximizing the total profit of rectangles packed into a rectangle. *Algorithmica*, 47(3):323–342, 2007. doi: 10.1007/S00453-006-0194-5. URL <https://doi.org/10.1007/s00453-006-0194-5>.
- Li Kong, Qi Qi, Yinyu Ye, and Zijie Zhou. Geometry-aware online scheduling for llm serving: From theoretical bound to system practice. *arXiv preprint arXiv:2606.22327*, 2026.
- Adam Kurpisz and Silvan Suter. Improved approximations for translational packing of convex polygons. In Inge Li Gørtz, Martin Farach-Colton, Simon J. Puglisi, and Grzegorz Herman, editors, *31st Annual European Symposium on Algorithms, ESA 2023, Amsterdam, The Netherlands, September 4-6, 2023*, volume 274 of *LIPIcs*, pages 76:1–76:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023. doi: 10.4230/LIPICS.ESA.2023.76. URL <https://doi.org/10.4230/LIPICS.ESA.2023.76>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In Jason Flinn, Margo I. Seltzer, Peter Druschel, Antoine Kaufmann, and Jonathan Mace, editors, *Proceedings of the 29th Symposium on Operating Systems*

- Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, pages 611–626. ACM, 2023. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- Shi Li. Scheduling to minimize total weighted completion time via time-indexed linear programming relaxations. *SIAM J. Comput.*, 49(4), 2020. doi: 10.1137/17M1156332. URL <https://doi.org/10.1137/17M1156332>.
- Andrea Lodi, Silvano Martello, and Michele Monaci. Two-dimensional packing problems: A survey. *Eur. J. Oper. Res.*, 141(2):241–252, 2002. doi: 10.1016/S0377-2217(02)00123-6. URL [https://doi.org/10.1016/S0377-2217\(02\)00123-6](https://doi.org/10.1016/S0377-2217(02)00123-6).
- Sasha Luccioni, Yacine Jernite, and Emma Strubell. Power hungry processing: Watts driving the cost of AI deployment? In *The 2024 ACM Conference on Fairness, Accountability, and Transparency, FAccT 2024, Rio de Janeiro, Brazil, June 3-6, 2024*, pages 85–99. ACM, 2024. doi: 10.1145/3630106.3658542. URL <https://doi.org/10.1145/3630106.3658542>.
- Rajeev Motwani, Steven J. Phillips, and Eric Torng. Non-clairvoyant scheduling. *Theor. Comput. Sci.*, 130(1):17–47, 1994. doi: 10.1016/0304-3975(94)90151-1. URL [https://doi.org/10.1016/0304-3975\(94\)90151-1](https://doi.org/10.1016/0304-3975(94)90151-1).
- Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In Dawn Song, Michael Carbin, and Tianqi Chen, editors, *Proceedings of the Sixth Conference on Machine Learning and Systems, MLSys 2023, Miami, FL, USA, June 4-8, 2023*. mlsys.org, 2023. URL https://proceedings.mlsys.org/paper_files/paper/2023/hash/c4be71ab8d24cdfb45e3d06dbfca2780-Abstract-mlsys2023.html.
- Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Saurabh Jha, Chen Wang, Hubertus Franke, Zbigniew T. Kalbarczyk, Tamer Basar, and Ravishankar K. Iyer. Efficient interactive LLM serving with proxy model-based sequence length prediction. *CoRR*, abs/2404.08509, 2024. doi: 10.48550/ARXIV.2404.08509. URL <https://doi.org/10.48550/arXiv.2404.08509>.
- Wayne E. Smith. Various optimizers for single-stage production. *Naval Research Logistics Quarterly*, 3(1-2):59–66, 1956. doi: 10.1002/nav.3800030106. URL <https://doi.org/10.1002/nav.3800030106>.
- A. Steinberg. A strip-packing algorithm with absolute performance bound 2. *SIAM J. Comput.*, 26(2):401–409, 1997. doi: 10.1137/S0097539793255801. URL <https://doi.org/10.1137/S0097539793255801>.
- Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Esha Choukse, Haoran Qiu, Rodrigo Fonseca, Josep Torrellas, and Ricardo Bianchini. TAPAS: thermal- and power-aware scheduling for LLM inference in cloud platforms. In Lieven Eeckhout, Georgios Smaragdakis, Katai Liang, Adrian Sampson, Martha A. Kim, and Christopher J. Rossbach, editors, *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2025, Rotterdam, Netherlands, 30 March 2025 - 3 April 2025*, pages 1266–1281. ACM, 2025a. doi: 10.1145/3676641.3716025. URL <https://doi.org/10.1145/3676641.3716025>.

- Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. Dynamollm: Designing LLM inference clusters for performance and energy efficiency. In *IEEE International Symposium on High Performance Computer Architecture, HPCA 2025, Las Vegas, NV, USA, March 1-5, 2025*, pages 1348–1362. IEEE, 2025b. doi: 10.1109/HPCA61900.2025.00102. URL <https://doi.org/10.1109/HPCA61900.2025.00102>.
- Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In Ada Gavrilovska and Douglas B. Terry, editors, *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, pages 173–191. USENIX Association, 2024. URL <https://www.usenix.org/conference/osdi24/presentation/sun-biao>.
- Rob van Stee and Johannes A. La Poutré. Minimizing the total completion time on-line on a single machine, using restarts. *J. Algorithms*, 57(2):95–129, 2005. doi: 10.1016/J.JALGOR.2004.10.001. URL <https://doi.org/10.1016/j.jalgor.2004.10.001>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, pages 5998–6008, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- Meixuan Wang, Yinyu Ye, and Zijie Zhou. LLM serving optimization with variable prefill and decode lengths. *CoRR*, abs/2508.06133, 2025. doi: 10.48550/ARXIV.2508.06133. URL <https://doi.org/10.48550/arXiv.2508.06133>.
- Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In Marcos K. Aguilera and Hakim Weatherspoon, editors, *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022, Carlsbad, CA, USA, July 11-13, 2022*, pages 521–538. USENIX Association, 2022. URL <https://www.usenix.org/conference/osdi22/presentation/yu>.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html.
- Zangwei Zheng, Xiaozhe Ren, Fuzhao Xue, Yang Luo, Xin Jiang, and Yang You. Response length perception and sequence scheduling: An llm-empowered LLM inference pipeline. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/ce7ff3405c782f761fac7f849b41ae9a-Abstract-Conference.html.

A Clairvoyant Scheduling Guarantees

The clairvoyant benchmark has three useful forms. Area-order greedy is nearly optimal when every job is small compared with the memory budget. For general instances, splitting jobs by full rectangle width and combining the two resulting schedules gives a constant approximation. A direct rectangle packing argument gives a smaller constant by using a known strip packing guarantee as a black box. The schedules below reserve the full rectangle width of each active job:

$$\ell_i \triangleq s_i + o_i,$$

i.e., reserving the full rectangle of each job.

A.1 Large-Memory Case

The large-memory case is a KV-cache analogue of the small-demand WSVF guarantee for capacitated machines [Cohen et al., 2024, Theorem 1].

Algorithm 5: Clairvoyant Area-Order Greedy (GREEDY)

Input: Job set $\mathcal{J}$ and memory budget M .

- 1 Order the jobs in $\mathcal{J}$ as $\pi(1), \dots, \pi(|\mathcal{J}|)$ so that $A_{\pi(1)} \leq \dots \leq A_{\pi(|\mathcal{J}|)}$;
 - 2 $\mathcal{B} \leftarrow \emptyset$ and $q \leftarrow 1$;
 - 3 **for** round $t = 0, 1, 2, \dots$ *until* ($q > |\mathcal{J}|$ and $\mathcal{B} = \emptyset$) **do**
 - 4 **while** $q \leq |\mathcal{J}|$ and $\sum_{i \in \mathcal{B}} \ell_i + \ell_{\pi(q)} \leq M$ **do**
 - 5 Start job $\pi(q)$ and add it to $\mathcal{B}$;
 - 6 $q \leftarrow q + 1$;
 - 7 Process every job in $\mathcal{B}$ for one round, and remove completed jobs from $\mathcal{B}$;
-

Theorem 4 (Bounded-width area-order greedy). *Fix $\varepsilon \in (0, 1)$. Suppose that every job in $\mathcal{J}$ satisfies*

$$\ell_i \leq \varepsilon M.$$

Then the schedule produced by GREEDY satisfies

$$\text{GREEDY}(\mathcal{J}) \leq \left(1 + \frac{2}{1 - \varepsilon}\right) \text{OPT}(\mathcal{J}).$$

Proof. Let $S_{\pi(r)}$ be the start time of job $\pi(r)$. Before time $S_{\pi(r)}$, if job $\pi(r)$ has not yet been admitted, then the next unstarted job has reserved width at most εM . Thus, after the greedy admission step in each such round, the active reserved width is larger than $(1 - \varepsilon)M$. All active jobs before time $S_{\pi(r)}$ belong to $\{\pi(1), \dots, \pi(r - 1)\}$. Therefore,

$$S_{\pi(r)} \leq \frac{1}{(1 - \varepsilon)M} \sum_{q=1}^{r-1} \ell_{\pi(q)} o_{\pi(q)}.$$

Since job $\pi(r)$ completes $o_{\pi(r)}$ rounds after it starts,

$$C_{\pi(r)} \leq o_{\pi(r)} + \frac{1}{(1 - \varepsilon)M} \sum_{q=1}^{r-1} \ell_{\pi(q)} o_{\pi(q)}.$$

Summing over all ranks gives

$$\text{GREEDY}(\mathcal{J}) \leq \sum_{i \in \mathcal{J}} o_i + \frac{1}{(1-\varepsilon)M} \sum_{q=1}^n (n-q) \ell_{\pi(q)} o_{\pi(q)}.$$

For every job i ,

$$\ell_i o_i = s_i o_i + o_i^2 \leq 2A_i.$$

Hence

$$\text{GREEDY}(\mathcal{J}) \leq \sum_{i \in \mathcal{J}} o_i + \frac{2}{(1-\varepsilon)M} \sum_{q=1}^n (n-q) A_{\pi(q)}.$$

Since π is the nondecreasing area order, Lemmas 1 and 3 give

$$\text{GREEDY}(\mathcal{J}) \leq \left(1 + \frac{2}{1-\varepsilon}\right) \text{OPT}(\mathcal{J}).$$

□

Corollary 2 (Large-memory area-order greedy). *If $\max_{i \in \mathcal{J}} \ell_i = o(M)$, then the clairvoyant area-order schedule in Theorem 4 is a $(3 + o(1))$ -approximation. Equivalently,*

$$\text{GREEDY}(\mathcal{J}) \leq (3 + o(1)) \text{OPT}(\mathcal{J}).$$

A.2 General Case via Black-Box Meta-Scheduling

For a general clairvoyant instance, view each job as a rectangle with width ℓ_i and height o_i . The jobs are separated by rectangle width. The small-width part uses GREEDY with the true full widths. The large-width part uses the same GREEDY routine on an auxiliary instance in which every job reserves rectangle width M . The two schedules are merged by the black-box meta-scheduler of Section 6.1.

Algorithm 6: Clairvoyant Width-Splitting Scheduler (CLAIRVOYANT-MIX)

Input: Job set $\mathcal{J}$, memory M , and threshold $\lambda \in (0, 1)$.

- 1 Set $\mathcal{J}_{\leq \lambda} \leftarrow \{i \in \mathcal{J} : \ell_i \leq \lambda M\}$;
 - 2 Set $\mathcal{J}_{> \lambda} \leftarrow \mathcal{J} \setminus \mathcal{J}_{\leq \lambda}$;
 - 3 Let $\mathcal{J}_{> \lambda}^M$ be $\mathcal{J}_{> \lambda}$ with reserved rectangle width M for each job;
 - 4 Run Algorithm 3 with sub-schedulers $\text{GREEDY}(\mathcal{J}_{\leq \lambda})$ and $\text{GREEDY}(\mathcal{J}_{> \lambda}^M)$;
-

Lemma 12 (Full-width auxiliary greedy call). *For $\lambda = 1/2$, the auxiliary instance $\mathcal{J}_{> \lambda}^M$ defined in Algorithm 6 satisfies*

$$\text{GREEDY}(\mathcal{J}_{> \lambda}^M) \leq 4 \text{OPT}(\mathcal{J}_{> \lambda}).$$

Proof. Set $m \triangleq |\mathcal{J}_{> \lambda}|$. Let $\pi(1), \dots, \pi(m)$ be the nondecreasing true area order of $\mathcal{J}_{> \lambda}$. In the auxiliary instance, every job reserves M and has area key $M o_i$. Thus GREEDY runs one job at a time in nondecreasing o_i . This order has no larger total completion time than the sequential schedule that follows the order π , so

$$\text{GREEDY}(\mathcal{J}_{> \lambda}^M) \leq \sum_{r=1}^m (m - r + 1) o_{\pi(r)}.$$

For every $i \in \mathcal{J}_{>\lambda}$,

$$A_i \geq \frac{\ell_i o_i}{2} > \frac{M o_i}{4}.$$

It follows that

$$\text{GREEDY}(\mathcal{J}_{>\lambda}^M) \leq \frac{4}{M} \sum_{r=1}^m (m - r + 1) A_{\pi(r)} \leq 4 \text{OPT}(\mathcal{J}_{>\lambda}),$$

where the last inequality follows from Lemma 1 applied to $\mathcal{J}_{>\lambda}$. $\square$

Theorem 5. *The schedule produced by CLAIRVOYANT-MIX satisfies*

$$\text{CLAIRVOYANT-MIX}(\mathcal{J}) \leq 28 \text{OPT}(\mathcal{J}).$$

Proof. For $\lambda = 1/2$, the same proof as Theorem 4 gives

$$\text{GREEDY}(\mathcal{J}_{\leq\lambda}) \leq 5 \text{OPT}(\mathcal{J}_{\leq\lambda}),$$

and Lemma 12 gives

$$\text{GREEDY}(\mathcal{J}_{>\lambda}^M) \leq 4 \text{OPT}(\mathcal{J}_{>\lambda}).$$

The branch-dependent bound in Lemma 10 gives a factor of 5 for the first branch and a factor of 7 for the second branch. Since Algorithm 6 places the small-width branch first, it gives

$$\text{CLAIRVOYANT-MIX}(\mathcal{J}) \leq 5 \text{GREEDY}(\mathcal{J}_{\leq\lambda}) + 7 \text{GREEDY}(\mathcal{J}_{>\lambda}^M).$$

Together with Lemma 2, this implies

$$\text{CLAIRVOYANT-MIX}(\mathcal{J}) \leq 25 \text{OPT}(\mathcal{J}_{\leq\lambda}) + 28 \text{OPT}(\mathcal{J}_{>\lambda}) \leq 28 \text{OPT}(\mathcal{J}).$$

This proves the theorem. $\square$

A.3 General Case with Direct Rectangle-Packing

This subsection gives a second clairvoyant bound using a stronger geometric primitive. The previous two subsections use area-order scheduling directly; here we call a strip packing algorithm as a black box, which gives a stronger makespan guarantee for each stage. The conversion from stage makespan guarantees to total completion time follows the doubling framework of the single-machine capacitated scheduling algorithm of [Cohen et al., 2024, Theorem 7.1]. Two simplifications apply in the present unweighted rectangle instance:

- The knapsack selection step for weighted rectangles becomes the exact prefix after sorting by rectangle area.
- For rectangles with width at most M , height at most L , and total area at most ML , the 2-approximation algorithm by Steinberg [1997] packs them in height at most $2L$.

These two changes give an 8-approximation for the rectangle instance, improving the $12 + \epsilon$ factor from the corresponding framework of Cohen et al. [2024]. The conversion from KV-cache jobs to rectangles loses another factor of 2 against the area lower bound, yielding the 16-approximation below.

Algorithm 7: Clairvoyant Rectangle-Packing Scheduler (PACKANDSCHEDULE)

Input: Job set $\mathcal{J}$ and memory M .

- 1 **for** stage $r = 0, 1, 2, \dots$ **until** all jobs have completed **do**
 - 2 Let $\mathcal{E}_r \leftarrow \{i \in \mathcal{J} : o_i \leq 2^r\}$, ordered by nondecreasing $\ell_i o_i$;
 - 3 Let $\mathcal{C}_r$ be the longest prefix of $\mathcal{E}_r$ whose total rectangle area is at most $M2^r$;
 - 4 Pack the rectangles $\{(\ell_i, o_i) : i \in \mathcal{C}_r\}$ in a M -width, 2^{r+1} -height strip by Steinberg's algorithm [Steinberg, 1997];
 - 5 Run the packed schedule ; // Completed jobs are treated as dummy jobs.
-

Theorem 6. *The schedule produced by PACKANDSCHEDULE satisfies*

$$\text{PACKANDSCHEDULE}(\mathcal{J}) \leq 16\text{OPT}(\mathcal{J}).$$

Proof. After stage r , the algorithm has completed at least $|\mathcal{C}_r|$ jobs by time

$$2 \sum_{q=0}^r 2^q < 4 \cdot 2^r.$$

The optimal schedule completes at most $|\mathcal{C}_r|$ jobs by time 2^{r-1} . To see that, let $\mathcal{C}_r^*$ be the jobs completed by an optimal schedule by that time. Every job in $\mathcal{C}_r^*$ has $o_i \leq 2^r$, and these jobs have total rectangle area at most

$$\sum_{i \in \mathcal{C}_r^*} \ell_i o_i \leq 2 \sum_{i \in \mathcal{C}_r^*} A_i \leq M2^r.$$

By the maximality of the prefix $\mathcal{C}_r$, this implies $|\mathcal{C}_r^*| \leq |\mathcal{C}_r|$.

For ranks $|\mathcal{C}_{r-1}|+1, \dots, |\mathcal{C}_r|$ with $r \geq 0$ (using the convention $\mathcal{C}_{-1} = \emptyset$), the algorithm completes these ranks by time less than $4 \cdot 2^r$, while the optimal schedule cannot complete these ranks before time 2^{r-2} . Summing over all ranks gives $\text{PACKANDSCHEDULE}(\mathcal{J}) \leq 16\text{OPT}$. $\square$

Remark 4. *The same argument improves the single-machine weighted capacitated scheduling bound of [Cohen et al., 2024, Theorem 7.1] from $12+\varepsilon$ to $8+\varepsilon$. It keeps their resource-augmented knapsack step, adds the natural scale eligibility condition on processing times ($\mathcal{E}_r$ in Algorithm 7), and replaces the factor-3 packing step based on Jansen and Zhang [2007] by Steinberg's factor-2 strip packing theorem.*

B Complete Non-Clairvoyant Rectangle Strip Scheduling

The rectangle strip guarantee in Theorem 2 assumes a useful width range. This section removes that assumption by mixing two rectangle strip schedulers through the black-box meta-scheduler.

Theorem 7. *For every rectangle strip instance with $w_i \leq M$ for all $i \in \mathcal{J}$, there is a non-clairvoyant scheduling algorithm whose total completion time is within a constant factor of the optimal clairvoyant total completion time:*

$$\text{RECTMIX}(\mathcal{J}) = O(1) \cdot \text{OPT}_{\text{rect}}(\mathcal{J}).$$

Algorithm 8: Complete Rectangle Strip Scheduler (RECTMIX)

- Input:** Rectangle job set $\mathcal{J}$, widths $\{w_i\}_{i \in \mathcal{J}}$, memory M , and threshold $\lambda \in (0, 1)$.
- 1 Set $\mathcal{J}_{\leq \lambda} \leftarrow \{i \in \mathcal{J} : w_i \leq \lambda M\}$;
 - 2 Set $\mathcal{J}_{> \lambda} \leftarrow \{i \in \mathcal{J} : w_i > \lambda M\}$;
 - 3 Run the black-box meta-scheduler with RECT on $\mathcal{J}_{\leq \lambda}$ and RECT on $\mathcal{J}_{> \lambda}$;
-

Proof. Run Algorithm 8 with $\lambda = (\sqrt{149} - 11)/2$. By Theorem 2, the two branch ratios are

$$4 + \frac{8}{1 - \lambda} \quad \text{and} \quad 4 + \frac{8}{\lambda}.$$

By Lemma 10, the first branch loses a factor of 5 and the second branch loses a factor of 7. Hence

$$\text{RECTMIX}(\mathcal{J}) \leq 5 \left(4 + \frac{8}{1 - \lambda}\right) \text{OPT}_{\text{rect}}(\mathcal{J}_{\leq \lambda}) + 7 \left(4 + \frac{8}{\lambda}\right) \text{OPT}_{\text{rect}}(\mathcal{J}_{> \lambda}).$$

The chosen λ makes both coefficients equal to $72 + 4\sqrt{149}$. Since $\mathcal{J}_{\leq \lambda}$ and $\mathcal{J}_{> \lambda}$ partition $\mathcal{J}$, Lemma 2 gives the theorem. $\square$

C Competitive Ratio for the Response-heavy Scheduler

This appendix proves the explicit constant used in Theorem 3. For the rest of the appendix, fix $\alpha = 2$ and a response-heavy job set $\mathcal{J}$; for every fixed $\alpha > 1$, the same argument gives the stated $O_\alpha(1)$ bound. Let $L \triangleq \lceil \log_2 M \rceil$. Let $\tau_{-1} = 0$ and define the phase classes

$$\mathcal{J}_r = \{i \in \mathcal{J} : \tau_{r-1} < o_i \leq \tau_r\}, \quad n_r = |\mathcal{J}_r|, \quad N_{\geq r} = \sum_{q=r}^L n_q, \quad N_{> r} = \sum_{q=r+1}^L n_q$$

for $r = 0, 1, \dots, L$. Let $\underline{\tau}_0 \triangleq 1$ and $\underline{\tau}_r \triangleq \tau_{r-1}$ for $r \geq 1$. Then every job $i \in \mathcal{J}_r$ satisfies

$$o_i \geq \underline{\tau}_r, \quad \tau_r \leq 2\underline{\tau}_r.$$

Lemma 13 (Phase efficiency). *For every phase r ,*

$$\frac{\tau_r}{k_r} \leq \frac{7}{2} \cdot \frac{\tau_r^2}{M}.$$

Proof. Write $\tau = \tau_r$ and $k = k_r$. If $\tau > M/2$, then $k = 1$, and the claim follows from $M < 2\tau$.

Assume $\tau \leq M/2$. Then the proxy prompt is $\bar{s}_r = \tau$. If $k = 1$, the maximality of k implies that $k = 2$ is infeasible. Thus

$$\text{Peak}(2, \tau, \tau) = 2\tau + \frac{3\tau + 2 - \gcd(\tau, 2)}{2} > M.$$

If τ is even, this gives $M < 7\tau/2$. If τ is odd, then $\text{Peak}(2, \tau, \tau) = (7\tau + 1)/2$ is an integer, so $M \leq (7\tau - 1)/2 < 7\tau/2$. In both cases, $\tau \leq (7/2)\tau^2/M$.

It remains to consider $k \geq 2$. By maximality, $k + 1$ is infeasible. Using $\gcd(\tau, k + 1) \geq 1$,

$$M < \text{Peak}(k + 1, \tau, \tau) \leq \tau(k + 1) + \frac{\tau(k + 1) + \tau + k}{2} = \frac{3k\tau + 4\tau + k}{2} \leq \frac{7k\tau}{2}.$$

The last inequality uses $k \geq 2$ and $\tau \geq 1$. Rearranging gives the claim. $\square$

Define the class-area lower-bound quantity

$$A_{\text{LB}} \triangleq \sum_{r=0}^L \tau_r^2 \left(\frac{n_r(n_r + 1)}{2} + n_r N_{>r} \right).$$

Lemma 14 (Class-area lower bound). *The optimum satisfies*

$$\text{OPT}(\mathcal{J}) \geq \frac{A_{\text{LB}}}{2M}.$$

Proof. For every class r and every job $i \in \mathcal{J}_r$, response dominance gives

$$A_i \geq \frac{o_i^2}{2} \geq \frac{\tau_r^2}{2}.$$

By Lemma 1, the lower-bound expression is

$$\frac{1}{M} \sum_{\ell=1}^{|\mathcal{J}|} (|\mathcal{J}| - \ell + 1) A_{\pi(\ell)},$$

where π orders jobs by nondecreasing area. This expression is the sum of all job areas plus, for every unordered pair of jobs, the smaller area in that pair, divided by M . Its diagonal and within-class pairs from class r contribute at least $\tau_r^2 n_r(n_r + 1)/(4M)$. The pairs with one job in class r and one job in a later class contribute at least $\tau_r^2 n_r N_{>r}/(2M)$. Summing these contributions over all classes gives $A_{\text{LB}}/(2M)$. $\square$

Theorem 3. *For every fixed $\alpha > 1$, the scheduler RESPONSE satisfies*

$$\text{RESPONSE}(\mathcal{J}) = O_\alpha(1) \text{OPT}(\mathcal{J})$$

on every response-heavy instance $\mathcal{J}$. In particular, for $\alpha = 2$,

$$\text{RESPONSE}(\mathcal{J}) \leq \frac{236}{3} \text{OPT}(\mathcal{J}) \approx 78.67 \text{OPT}(\mathcal{J}).$$

Proof. Let D_r be the actual elapsed time of the SPS call in phase r . By the start-offset rule of SPS, the last start time in an SPS call on at most $N_{\geq r}$ jobs is less than $\tau_r N_{\geq r}/k_r$, so

$$D_r \leq \tau_r + \frac{\tau_r}{k_r} N_{\geq r}.$$

By Lemma 9, every job in classes $r, r + 1, \dots, L$ is unfinished before phase r , so

$$\text{RESPONSE}(\mathcal{J}) \leq \sum_{r=0}^L D_r N_{\geq r} \leq \sum_{r=0}^L \tau_r N_{\geq r} + \sum_{r=0}^L \frac{\tau_r}{k_r} N_{\geq r}^2.$$

For the linear term, geometric growth gives $\sum_{r=0}^{\ell} \tau_r \leq 4\tau_\ell$ for every class ℓ . Therefore

$$\sum_{r=0}^L \tau_r N_{\geq r} = \sum_{\ell=0}^L n_\ell \sum_{r=0}^{\ell} \tau_r \leq 4 \sum_{\ell=0}^L n_\ell \tau_\ell \leq 4 \sum_{i \in \mathcal{J}} o_i \leq 4 \text{OPT}(\mathcal{J}).$$

Set $c = 7/2$. By Lemma 13,

$$\sum_{r=0}^L \frac{\tau_r}{k_r} N_{\geq r}^2 \leq \frac{c}{M} \sum_{r=0}^L \tau_r^2 N_{\geq r}^2.$$

Expanding over ordered pairs of jobs and charging each pair to the smaller response class gives

$$\sum_{r=0}^L \tau_r^2 N_{\geq r}^2 \leq \frac{16}{3} \sum_{\ell, s=0}^L n_\ell n_s \tau_{\min\{\ell, s\}}^2 \leq \frac{32}{3} A_{\text{LB}}.$$

The first inequality uses $\sum_{r=0}^\ell \tau_r^2 \leq (16/3) \tau_\ell^2$ for every class ℓ . The second inequality compares ordered pairs with the definition of A_{LB} : within a class, $n_\ell^2 \leq n_\ell(n_\ell + 1)$, and between two different classes the ordered-pair count is twice the unordered-pair count.

Combining the linear and quadratic bounds gives

$$\text{RESPONSE}(\mathcal{J}) \leq 4 \text{OPT}(\mathcal{J}) + \frac{112}{3M} A_{\text{LB}}.$$

By Lemma 14, $A_{\text{LB}} \leq 2M \text{OPT}(\mathcal{J})$. Hence

$$\text{RESPONSE}(\mathcal{J}) \leq \left(4 + \frac{224}{3}\right) \text{OPT}(\mathcal{J}) = \frac{236}{3} \text{OPT}(\mathcal{J}) \approx 78.67 \text{OPT}(\mathcal{J}).$$

□

Lemma 15 (Response subset control). *Fix a response-heavy set $\mathcal{R}^*$ and a deterministic order used by RESPONSE. For each $i \in \mathcal{R}^*$, let C_i^{R} be the completion time of i in $\text{RESPONSE}(\mathcal{R}^*)$. Then, for every $\mathcal{J}' \subseteq \mathcal{R}^*$ and every $i \in \mathcal{J}'$, job i completes in $\text{RESPONSE}(\mathcal{J}')$ by time C_i^{R} .*

Proof. Fix any job $i \in \mathcal{J}'$, and let r be the first phase with $\tau_r \geq o_i$. Run $\text{RESPONSE}(\mathcal{J}')$ using the order induced by the fixed order on $\mathcal{R}^*$. By Lemma 9, job i completes in phase r in both $\text{RESPONSE}(\mathcal{J}')$ and $\text{RESPONSE}(\mathcal{R}^*)$. For every phase $q < r$, whether a job is considered in that phase depends only on its prompt length, response length, and the cap sequence. Thus the jobs considered in the run on $\mathcal{J}'$ form a subsequence of the jobs considered in the run on $\mathcal{R}^*$. For every remaining job in this subsequence, its rank is no larger than its rank in the run on $\mathcal{R}^*$. Since the SPS start offset is $\lfloor j\tau_q/k_q \rfloor$ for rank j , each remaining job starts no later in the run on $\mathcal{J}'$ than in the run on $\mathcal{R}^*$. The real running time of every remaining job in phase q is unchanged, so the duration of phase q in the run on $\mathcal{J}'$ is no larger than the duration of phase q in the run on $\mathcal{R}^*$. The same subsequence property holds inside phase r : the rank of job i in the run on $\mathcal{J}'$ is at most its rank in the run on $\mathcal{R}^*$, so the SPS start offset of job i is no later in the run on $\mathcal{J}'$ than in the run on $\mathcal{R}^*$. After job i starts in phase r , the remaining time until completion is o_i in both runs. Combining these components gives the claim. □

D Alternative Objectives

This appendix gives two extensions of the meta-scheduling framework.

- Section D.1 shows that the same algorithm is also constant-competitive for makespan.

- Section D.2 extends the routing framework to total completion time with online arrivals.

For online arrivals, the natural latency objective is total flow time. Under (oblivious) adversarial arrivals, however, online KV-cache scheduling admits an $\Omega(\sqrt{n})$ lower bound on the competitive ratio for total flow time, even in the identical-prompt special case [Jaillet et al., 2025, Feng et al., 2026]. Total completion time is the closest feasible alternative: for a fixed arrival instance, the two objectives induce the same optimal schedules. Online completion-time objectives have also been studied in classical scheduling [Epstein and van Stee, 2003, van Stee and Poutré, 2005].

D.1 Makespan

For a job set $\mathcal{J}' \subseteq \mathcal{J}$, let $\text{OPT}_{\text{span}}(\mathcal{J}')$ be the optimal clairvoyant makespan for scheduling exactly the jobs in $\mathcal{J}'$. For a schedule produced by an algorithm ALG , let $\text{MAKESPAN}[\text{ALG}(\mathcal{J}')]$ denote its makespan.

Lemma 16 (Makespan lower bounds). *For every $\mathcal{J}' \subseteq \mathcal{J}$,*

$$\text{OPT}_{\text{span}}(\mathcal{J}') \geq \max_{i \in \mathcal{J}'} o_i, \quad \text{OPT}_{\text{span}}(\mathcal{J}') \geq \frac{1}{M} \sum_{i \in \mathcal{J}'} A_i.$$

The first bound is the processing requirement of the longest job. The second follows because all memory-time areas of jobs must be supplied. We use the branch sets and capped small instance from Section 6.2.

Proposition 4 (Branch makespan bounds). *Set $\Lambda = M/4$ and $\alpha = 2$. Then*

$$\text{MAKESPAN}[\text{RECT}(\mathcal{L})], \quad \text{MAKESPAN}[\text{RECT}(\hat{\mathcal{S}})], \quad \text{MAKESPAN}[\text{RESPONSE}(\mathcal{R}^*)]$$

are all $O(1) \cdot \text{OPT}_{\text{span}}(\mathcal{J})$.

Proof. For either rectangle branch instance $\mathcal{J}'$, Lemma 5 implies, after taking the maximum over jobs in $\mathcal{J}'$,

$$\text{MAKESPAN}[\text{RECT}(\mathcal{J}')] = O \left(\max_{i \in \mathcal{J}'} p_i + \frac{1}{M} \sum_{i \in \mathcal{J}'} A_i^{\text{RECT}} \right),$$

using the same width-range conditions as in the proofs of Propositions 2 and 3. For the large branch, $A_i^{\text{RECT}} \leq 4A_i$. For the prompt-heavy branch, the capped rectangle area is at most $2A_i$. Thus both rectangle branch makespans are $O(1) \cdot \text{OPT}_{\text{span}}(\mathcal{J})$ by Lemma 16.

For the response-heavy branch, the phase-duration estimate used in the proof of Theorem 3, summed over geometric response caps, gives

$$\text{MAKESPAN}[\text{RESPONSE}(\mathcal{R}^*)] = O \left(\max_{i \in \mathcal{R}^*} o_i + \frac{1}{M} \sum_{i \in \mathcal{R}^*} o_i^2 \right).$$

Here the geometric caps bound the linear contribution by the largest response length, and the phase-efficiency term contributes the quadratic area term. Since every job in $\mathcal{R}^*$ is response-heavy, $o_i^2 \leq 2A_i$. Another application of Lemma 16 completes the proof. $\square$

The following theorem gives the makespan extension.

Theorem 8. *For every feasible batch KV-cache scheduling instance with arbitrary prompt lengths and arbitrary response lengths, there is a fully non-clairvoyant scheduling algorithm whose makespan is within a constant factor of the optimal clairvoyant makespan:*

$$\text{MAKESPAN}[\text{ALG}(\mathcal{J})] = O(1) \cdot \text{OPT}_{\text{span}}(\mathcal{J}).$$

Proof. Set $\Lambda = M/4$ and $\alpha = 2$, and run the routing meta-scheduler ROUTE in Algorithm 4. Let

$$T = \max \left\{ \max_{i \in \mathcal{L}} C_i^L, \max_{i \in \mathcal{S}} C_i^P, \max_{i \in \mathcal{R}^*} C_i^R \right\}.$$

By the subset-control discussion in Section 6.2, every current branch call completes or certifies its remaining jobs within the corresponding reference bound. Choose the first stage r with $2^r \geq T$. In this stage, the prompt-heavy branch call completes all remaining jobs in $\mathcal{P}^*$ and certifies all remaining jobs in $\mathcal{R}^*$. The certified jobs then move to $\mathcal{R}$, and the following response-heavy branch call completes them; the large branch call completes all large jobs. Thus all jobs finish by the end of the stage. The total time through that stage is $O(2^r) = O(T)$. The same bounds used in Proposition 4 give $T = O(1) \cdot \text{OPT}_{\text{span}}(\mathcal{J})$, which proves the theorem. $\square$

D.2 Total Completion Time with Online Arrivals

Each job i now has an arrival time $a_i \in \mathbb{N}$. Before time a_i , job i is not visible to the scheduler. At time a_i , its prompt length s_i becomes known, while its response length o_i is still unknown until completion. Let $\text{OPT}_{\text{arr}}(\mathcal{J})$ be the optimal total completion time of a clairvoyant schedule that knows all jobs and response lengths in advance, but may process job i only after time a_i .

Lemma 17 (Online-arrival lower bounds).

$$\text{OPT}_{\text{arr}}(\mathcal{J}) \geq \text{OPT}(\mathcal{J}), \quad \text{OPT}_{\text{arr}}(\mathcal{J}) \geq \sum_{i \in \mathcal{J}} (a_i + o_i).$$

The first bound follows by removing arrival constraints. The second follows because job i cannot complete before time $a_i + o_i$.

The online scheduler uses the same pool convention: newly arrived jobs enter either $\mathcal{L}$ or $\mathcal{P}$, certified but unfinished jobs move to $\mathcal{R}$, and completed jobs are removed from their branch pools.

The following theorem gives the online-arrival extension.

Theorem 9. *For every feasible online-arrival KV-cache scheduling instance with arbitrary prompt lengths, arbitrary response lengths, and arbitrary arrival times, there is an online non-clairvoyant scheduling algorithm whose total completion time is within a constant factor of the optimal clairvoyant total completion time among schedules that process each job only after its arrival:*

$$\text{ROUTEONLINE}(\mathcal{J}) = O(1) \cdot \text{OPT}_{\text{arr}}(\mathcal{J}).$$

Proof. Set $\Lambda = M/4$ and $\alpha = 2$, and run the online-arrival routing meta-scheduler in Algorithm 9. The online arrival constraint only delays the time at which a job can enter the routing process. A job arriving at time a_i is admitted at the start of the first stage that begins at or after a_i ; by the doubling schedule, the elapsed time before that stage is $O(a_i)$. After this prefix, the same routing calculation as in the batch proof applies to job i , using the reference bounds C_i^L, C_i^P, C_i^R as in

Algorithm 9: Online-Arrival Routing Meta-Scheduler (ROUTEONLINE)

Input: Memory M and prompt cutoff Λ .

Online Arrival: Job i becomes visible at time a_i and reveals its prompt length s_i .

```
1  $\mathcal{L} \leftarrow \emptyset$ ; // large branch
2  $\mathcal{P} \leftarrow \emptyset$ ; // prompt-heavy branch
3  $\mathcal{R} \leftarrow \emptyset$ ; // response-heavy branch
4 for stage  $r = 0, 1, 2, \dots$  do
    // Admit new jobs.
5   for each newly arrived job that has not been assigned to a branch do
6     if  $s_i > \Lambda$  then
7       | Add  $i$  to  $\mathcal{L}$ ;
8     else
9       | Add  $i$  to  $\mathcal{P}$ ;
    // Run current doubling stage.
10  Run a fresh copy of the prompt-heavy branch scheduler on  $\mathcal{P}$  for  $2^r$  rounds;
11  Move every newly certified job from  $\mathcal{P}$  to  $\mathcal{R}$ ;
12  Run a fresh copy of the response-heavy branch scheduler on  $\mathcal{R}$  for  $2^r$  rounds;
13  Run a fresh copy of the large branch scheduler on  $\mathcal{L}$  for  $2^r$  rounds;
```

the proof of Theorem 1. The current branch calls are on subsets of the corresponding reference instances, so the same subset arguments apply. Thus

$$\begin{aligned} C_i^{\text{ROUTEONLINE}} &= O(a_i + C_i^{\text{L}}) && \text{for every } i \in \mathcal{L}, \\ C_i^{\text{ROUTEONLINE}} &= O(a_i + C_i^{\text{P}}) && \text{for every } i \in \mathcal{P}^*, \\ C_i^{\text{ROUTEONLINE}} &= O(a_i + C_i^{\text{P}} + C_i^{\text{R}}) && \text{for every } i \in \mathcal{R}^*. \end{aligned}$$

Summing over all jobs gives the same three branch sums as in the proof of Theorem 1, plus the arrival term $\sum_{i \in \mathcal{J}} a_i$. The branch sums are bounded by $O(1) \cdot \text{OPT}(\mathcal{J})$, while the arrival term is bounded by Lemma 17 and $\text{OPT}(\mathcal{J}) \leq \text{OPT}_{\text{arr}}(\mathcal{J})$. Therefore $\text{ROUTEONLINE}(\mathcal{J}) = O(1) \cdot \text{OPT}_{\text{arr}}(\mathcal{J})$. $\square$